



Products Admin Tiered Editor (PR 3 of 3) — Implementation Plan

Quick View / Features / Technical / SEO & Links tabs, publish validation, drag reorder & categories, and a real-component draft preview — on the PR 1 schema and PR 2 public UX

Date: 2026-07-05
Status: Implementation plan (PR 3 of 3)
Owner: Fatima Abdul Raheem (CEO)
Branch: feat/products-admin

Contents

1	Goal	2
2	Architecture	2
3	Global Constraints	2
4	File Structure	3
5	Task 1: Data layer + server actions (TDD on the pure logic)	4
6	Task 2: Tiered editor UI (Quick View / Features / Technical / SEO & Links)	20
7	Task 3: List view + categories manager + preview route	43
8	Task 4: Visual QA, plan PDF twin, PR	55
9	Self-Review (done)	56

1 Goal

For agentic workers: REQUIRED SUB-SKILL — use `superpowers:subagent-driven-development` (recommended) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

The complete products admin on top of the PR 1 schema and PR 2 public UX — a tiered editor (Quick View / Features / Technical / SEO & Links tabs, every field labeled with its public destination), publish validation that also guards edits to already-published rows (with the “Unpublish & save as draft” escape), image uploads to the `product-images` bucket, a drag-reorderable + filterable list with Duplicate, an admin-managed categories page with a delete guard, and a preview route that renders the **actual** public components from the last-saved draft.

2 Architecture

The editor is one client island holding a single form object (plus keyed lists for drag-and-drop) and submits a **JSON payload** to one server action, `saveProductFull` — nested lists (highlights, features, related-post links) don’t fit flat `FormData`, and one action keeps the product + child-table writes together. Child rows (`product_features`, `product_blog_links`) are replaced wholesale on save (delete + insert; volumes are tiny, ordering writes become trivial). Publish validation is a **pure helper** used twice: client-side for instant feedback (Publish button state + checklist) and server-side as the authority. Drag-and-drop is three shared primitives (`SortableList/SortableItem` over `@dnd-kit`) reused by five surfaces: highlights, features, related posts, the product list, and the categories manager. The preview route fetches with the admin server client, maps through the **same** `toProductDetail` the public routes use, and renders the untouched `ProductDetailView / ProductQuickView` — zero changes to public components. Uploads go through a server action (admin session + storage RLS), which requires bumping Next’s server-action body limit above the default 1 MB.

Tech Stack: Next.js 16 App Router, React 19, Tailwind v4, Supabase (server client as the signed-in admin; RLS is the write boundary), `@dnd-kit`, Vitest + Testing Library.

3 Global Constraints

- Brand: **NurvexThink** / `nurvexthink`. Conventional Commits. Branch: `feat/products-admin` (from current main).
- **The ONLY new dependencies are** `@dnd-kit/core`, `@dnd-kit/sortable`, `@dnd-kit/utilities` (spec §6 sanctions exactly this: accessible keyboard + touch DnD).
- **No changes to public components or queries** — `product-quick-view.tsx`, `product-detail-view.tsx`, `product-card.tsx`, `products-explorer.tsx`, and everything in `src/lib/queries.ts` are imported, never edited.
- Admin auth: the `(panel)` layout already redirects signed-out users (`src/app/admin/(panel)/layout.tsx`); server actions run as the signed-in admin via `createServerSupabaseClient()` and **RLS is the write boundary** (`private.is_admin()` policies from 0001/0003). `service_role` is never used.
- **Every mutation revalidates** `/`, `/products`, the affected `/products/[slug]` path(s), and `/admin/products` (the established `revalidateAll` pattern, extended).
- Spec contract: `docs/superpowers/specs/2026-07-04-products-experience-design.md` §3 (Admin UX), §8.3.
- Publish validation (verbatim from spec §3): the Quick View tier — **name, tagline, summary**, **≥ 3 highlights, cover image, category** — must be complete for a product to be **or remain** published. Publish enables only when valid; saving edits to a published product re-runs the

same validation and an invalid save is blocked with the *Unpublish & save as draft* option; drafts save anytime, half-finished.

- Duplicate semantics (verbatim from spec §3): new **draft**; slug `-copy / -copy-2` with a uniqueness retry; featured reset to false; sort_order = end of list; features + related-post links copied; **image URLs copied by reference** (shared Storage objects — future delete-cleanup must check referrers; already flagged in the spec).
- Reordering is **disabled while any filter is active** (positions are global) and the list groups featured items first to mirror the public order.
- Uploads: product-images bucket, content-type image/*, **≤ 5 MB**, via a server action (needs the `serverActions.bodySizeLimit` bump in `next.config.ts` — Next's default is 1 MB).
- Server Components by default; "use client" only at leaves: the editor, the list table, the categories manager, upload fields, sortable primitives, and the preview quick-view wrapper (CLAUDE.md).
- Definition of green: `npm run typecheck && npm run lint && npm test && npm run build` all exit 0 — **after every task**.

4 File Structure

- `next.config.ts` — **modify**: `serverActions.bodySizeLimit: "6mb"` so ≤ 5 MB image uploads survive multipart overhead.
- `src/lib/product-admin.ts` — **create**: pure helpers — publish-tier validation, slugify, duplicate-slug candidates, sort-order math. `src/lib/product-admin.test.ts` — **create** (TDD).
- `src/lib/admin-queries.ts` — **modify** (full rewrite): public-mirroring product order + category join, `getProductFullAdmin` (features + links + post status), `listCategoriesAdmin`, `countProductsByCategory`, `listPostsForPicker`; `blog/orders` helpers unchanged.
- `src/app/admin/(panel)/products/actions.ts` — **modify** (full rewrite): `saveProductFull` (JSON payload, tier validation, child-row replace), `reorderProducts`, `duplicateProduct`, `uploadProductImage`, `deleteProduct`; legacy form actions kept temporarily so old consumers compile, removed in Tasks 2–3.
- `src/app/admin/(panel)/products/categories/actions.ts` — **create**: category CRUD + reorder with the referenced-delete guard.
- `src/components/admin/form-styles.ts` — **create**: the shared `fieldClass/labelClass/hintClass` strings (previously private to `product-form.tsx`).
- `src/components/admin/sortable-list.tsx` — **create** ("use client"): `SortableList` + `SortableItem` @dnd-kit primitives.
- `src/components/admin/image-upload.tsx` — **create** ("use client"): `ImageUploadField` (single, with preview/replace/remove) + `GalleryUploadField` (multi with remove).
- `src/components/admin/related-posts-picker.tsx` — **create** ("use client"): search-filter over all posts, multi-select, drag-order, draft badge.
- `src/components/admin/product-editor.tsx` — **create** ("use client"): the tiered editor. `src/components/admin/product-editor.test.tsx` — **create**.
- `src/components/admin/products-table.tsx` — **create** ("use client"): list rows, filters, grouped drag reorder, Duplicate.
- `src/components/admin/categories-manager.tsx` — **create** ("use client"): add / rename / delete-guarded / reorder.

- `src/components/admin/preview-quick-view.tsx` — **create** ("use client"): mounts `ProductQuickView` open, `onClose` → `router.back()`.
- `src/app/admin/(panel)/products/page.tsx` — **modify** (rewrite): renders `ProductsTable`.
- `src/app/admin/(panel)/products/new/page.tsx, .../[id]/page.tsx` — **modify** (rewrite): render `ProductEditor` with categories + posts.
- `src/app/admin/(panel)/products/categories/page.tsx` — **create**: categories manager route.
- `src/app/admin/(panel)/products/[id]/preview/page.tsx` — **create**: both-tier preview from the last-saved draft.
- `src/components/admin/product-form.tsx` — **delete** (Task 2; replaced by the tiered editor).

5 Task 1: Data layer + server actions (TDD on the pure logic)

Files

- Create: `src/lib/product-admin.ts`, `src/lib/product-admin.test.ts`
- Modify: `src/lib/admin-queries.ts` (full-file replace)
- Modify: `src/app/admin/(panel)/products/actions.ts` (full-file replace)
- Create: `src/app/admin/(panel)/products/categories/actions.ts`
- Modify: `next.config.ts`

Interfaces

- Consumes: `ProductRow`, `ProductCategoryRow`, `BlogPostRow` from `src/lib/supabase/types.ts`; `createServerSupabaseClient` from `src/lib/supabase/server.ts`.
- Produces (later tasks rely on these exact names):
 - `src/lib/product-admin.ts`: `validatePublishTier(f: PublishTierFields): { valid: boolean; missing: string[] }, slugify(text), copySlug(slug, attempt), nextSortOrder(existing: number[]), sortOrderSequence(count)`.
 - `src/lib/admin-queries.ts`: **types** `AdminProductRow`, `AdminFeatureJoin`, `AdminLinkJoin`, `AdminProductFull`, `PostPickerItem`; **functions** `listProductsAdmin()` (ordered featured DESC, sort_order ASC, created_at DESC — mirrors public), `getProductFullAdmin(id)`, `listCategoriesAdmin()`, `countProductsByCategory()`, `listPostsForPicker()`.
 - `actions.ts`: **types** `ProductPayload`, `EditorFeature`, `SaveIntent = "save" | "publish" | "unpublish"`, `SaveProductResult`, `MutationResult`, `UploadResult`; **actions** `saveProductFull(id | null, intent, payload)`, `reorderProducts(orderedIds)`, `duplicateProduct(id)`, `uploadProduct(formData)`, `deleteProduct(formData)`.
 - `categories/actions.ts`: `createCategory(name)`, `renameCategory(id, name)`, `deleteCategory(id)`, `reorderCategories(orderedIds)` — **all return** `CategoryActionResult = { ok: boolean; error: string | null }`.

Step 1: Branch

```
git checkout main && git pull && git checkout -b feat/products-admin
```

Step 2: Write failing tests

Create `src/lib/product-admin.test.ts`:

```
import { describe, it, expect } from "vitest";
import {
  validatePublishTier,
  slugify,
  copySlug,
  nextSortOrder,
  sortOrderSequence,
} from "@lib/product-admin";

const complete = {
  name: "FluxBoard",
  tagline: "Standups into shipped work.",
  summary: "A keyboard-first project board.",
  highlights: ["Fast", "Realtime", "Keyboard-first"],
  coverImage: "https://cdn.example/cover.png",
  categoryId: "11111111-1111-1111-1111-111111111111",
};

describe("validatePublishTier", () => {
  it("passes a complete quick-view tier", () => {
    expect(validatePublishTier(complete)).toEqual({ valid: true, missing: [] });
  });

  it("names every missing field, in editor order", () => {
    const check = validatePublishTier({
      name: "",
      tagline: "",
      summary: "",
      highlights: [],
      coverImage: null,
      categoryId: null,
    });
    expect(check.valid).toBe(false);
    expect(check.missing).toEqual([
      "Name",
      "Tagline",
      "Summary",
      "At least 3 highlights",
      "Cover image",
      "Category",
    ]);
  });

  it("does not count blank highlights toward the minimum of 3", () => {
    const check = validatePublishTier({ ...complete, highlights: ["Fast", " ", "Realtime"] });
    expect(check.missing).toEqual(["At least 3 highlights"]);
  });

  it("treats whitespace-only text fields as missing", () => {
    expect(validatePublishTier({ ...complete, tagline: " " }).missing).toEqual(["Tagline"]);
  });
});

describe("slugify", () => {
```

```

    it("lowercases and dashes non-alphanumerics", () => {
      expect(slugify("Flux Board 2.0!")).toBe("flux-board-2-0");
    });

    it("trims leading/trailing dashes", () => {
      expect(slugify(" --Vault-- ")).toBe("vault");
    });
  });

describe("copySlug", () => {
  it("suffixes -copy, then -copy-2, -copy-3...", () => {
    expect(copySlug("fluxboard", 1)).toBe("fluxboard-copy");
    expect(copySlug("fluxboard", 2)).toBe("fluxboard-copy-2");
    expect(copySlug("fluxboard", 3)).toBe("fluxboard-copy-3");
  });

  it("never stacks -copy when duplicating a duplicate", () => {
    expect(copySlug("fluxboard-copy", 1)).toBe("fluxboard-copy");
    expect(copySlug("fluxboard-copy", 2)).toBe("fluxboard-copy-2");
    expect(copySlug("fluxboard-copy-2", 3)).toBe("fluxboard-copy-3");
  });
});

describe("nextSortOrder", () => {
  it("starts at 10 and continues past the max in steps of 10", () => {
    expect(nextSortOrder([])).toBe(10);
    expect(nextSortOrder([10, 30, 20])).toBe(40);
  });
});

describe("sortOrderSequence", () => {
  it("yields 10, 20, 30 ...for n rows", () => {
    expect(sortOrderSequence(3)).toEqual([10, 20, 30]);
    expect(sortOrderSequence(0)).toEqual([]);
  });
});

```

Step 3: Run to verify failure

Run: `npx vitest run src/lib/product-admin.test.ts` — expect FAIL (Cannot find module '@/lib/product-admin').

Step 4: Create `src/lib/product-admin.ts`

```

/**
 * Pure helpers for the products admin (spec §3). No I/O --unit-tested.
 * validatePublishTier runs twice per save: client-side for instant feedback,
 * server-side (saveProductFull) as the authority.
 */

export type PublishTierFields = {
  name: string;
  tagline: string;
  summary: string;
  highlights: string[];
  coverImage: string | null;
  categoryId: string | null;
};

```

```

export type PublishCheck = { valid: boolean; missing: string[] };

/**
 * Spec §3: the Quick View tier must be complete for a product to be --or
 * remain --published. Labels are shown verbatim in the editor's error banner.
 */
export function validatePublishTier(f: PublishTierFields): PublishCheck {
  const missing: string[] = [];
  if (!f.name.trim()) missing.push("Name");
  if (!f.tagline.trim()) missing.push("Tagline");
  if (!f.summary.trim()) missing.push("Summary");
  if (f.highlights.filter((h) => h.trim()).length < 3) missing.push("At least 3
    highlights");
  if (!f.coverImage) missing.push("Cover image");
  if (!f.categoryId) missing.push("Category");
  return { valid: missing.length === 0, missing };
}

/** Same shape the 0003 migration used to slugify category names. */
export function slugify(text: string): string {
  return text
    .toLowerCase()
    .replace(/[~a-z0-9]+/g, "-")
    .replace(/^-+|-+$/g, "");
}

/**
 * Duplicate-slug candidates (spec §3): '-copy', then '-copy-2', '-copy-3'...
 * Duplicating a duplicate re-uses the original base (no '-copy-copy').
 */
export function copySlug(slug: string, attempt: number): string {
  const base = slug.replace(/-copy(-\d+)?$/, "");
  return attempt <= 1 ? `${base}-copy` : `${base}-copy-${attempt}`;
}

/** Next end-of-list sort_order --keeps 0003's convention of gaps of 10. */
export function nextSortOrder(existing: number[]): number {
  return existing.length === 0 ? 10 : Math.max(...existing) + 10;
}

/** 10, 20, 30 ...for a full reorder write. */
export function sortOrderSequence(count: number): number[] {
  return Array.from({ length: count }, (_, i) => (i + 1) * 10);
}

```

Step 5: Run tests — pass

Run: `npx vitest run src/lib/product-admin.test.ts` → all green (10 tests).

Step 6: Replace `src/lib/admin-queries.ts`

Whole file — `blog/orders` helpers reproduced unchanged; `getProductAdmin` kept as legacy until Task 2 rewrites the edit page:

```

import { createServerSupabaseClient } from "@lib/supabase/server";
import type {
  ProductRow,
  ProductCategoryRow,

```



```

    BlogPostRow,
    OrderRow,
  } from "@lib/supabase/types";

  // These run as the logged-in admin, so the "admin reads all" RLS policies return
  // every row (including unpublished products and draft posts).

  /** List row: product + category name for the admin table. */
  export type AdminProductRow = ProductRow & {
    product_categories: { name: string } | null;
  };

  /** Feature/link join shapes --structurally compatible with queries.ts's
   * ProductRowJoined, so toProduct/toProductDetail accept AdminProductFull. */
  export type AdminFeatureJoin = {
    id: string;
    title: string;
    description: string | null;
    image: string | null;
    sort_order: number;
  };

  export type AdminLinkJoin = {
    blog_post_id: string;
    sort_order: number;
    blog_posts: {
      slug: string;
      title: string;
      excerpt: string | null;
      cover_image: string | null;
      status: "draft" | "published";
    } | null;
  };

  export type AdminProductFull = AdminProductRow & {
    product_features: AdminFeatureJoin[];
    product_blog_links: AdminLinkJoin[];
  };

  const ADMIN_PRODUCT_FULL_SELECT =
    `*, product_categories(name), product_features(id,title,description,image,sort_order),
    product_blog_links(blog_post_id, sort_order, blog_posts(slug,title,excerpt,
    cover_image,status))`;

  /** Ordered like the public grid (spec §2) so the admin list predicts it. */
  export async function listProductsAdmin(): Promise<AdminProductRow[]> {
    const supabase = await createServerSupabaseClient();
    const { data } = await supabase
      .from("products")
      .select("*, product_categories(name)")
      .order("featured", { ascending: false })
      .order("sort_order", { ascending: true })
      .order("created_at", { ascending: false });
    // Hand-rolled Database type has no Relationships metadata, so the select-string
    // parser can't type nested joins --cast to the known shape.
    return (data ?? []) as unknown as AdminProductRow[];
  }

  /** Product + features + links (+ post status for the draft badge / preview filter). */
  export async function getProductFullAdmin(id: string): Promise<AdminProductFull | null>

```

```

    {
    const supabase = await createServerSupabaseClient();
    const { data } = await supabase
      .from("products")
      .select(ADMIN_PRODUCT_FULL_SELECT)
      .eq("id", id)
      .maybeSingle();
    return (data as unknown as AdminProductFull | null) ?? null;
  }

  // Legacy --removed in Task 2 when the edit page switches to getProductFullAdmin.
  export async function getProductAdmin(id: string): Promise<ProductRow | null> {
    const supabase = await createServerSupabaseClient();
    const { data } = await supabase.from("products").select("*").eq("id", id).maybeSingle()
      ;
    return data ?? null;
  }

  export async function listCategoriesAdmin(): Promise<ProductCategoryRow[]> {
    const supabase = await createServerSupabaseClient();
    const { data } = await supabase
      .from("product_categories")
      .select("*")
      .order("sort_order", { ascending: true })
      .order("name", { ascending: true });
    return data ?? [];
  }

  /** category_id →number of products referencing it (delete guard + manager UI). */
  export async function countProductsByCategory(): Promise<Record<string, number>> {
    const supabase = await createServerSupabaseClient();
    const { data } = await supabase.from("products").select("category_id");
    const counts: Record<string, number> = {};
    for (const row of data ?? []) {
      if (row.category_id) counts[row.category_id] = (counts[row.category_id] ?? 0) + 1;
    }
    return counts;
  }

  /** Everything the related-posts picker needs --drafts included (they're badged). */
  export type PostPickerItem = Pick<BlogPostRow, "id" | "title" | "slug" | "status">;

  export async function listPostsForPicker(): Promise<PostPickerItem[]> {
    const supabase = await createServerSupabaseClient();
    const { data } = await supabase
      .from("blog_posts")
      .select("id,title,slug,status")
      .order("created_at", { ascending: false });
    return data ?? [];
  }

  export async function listPostsAdmin(): Promise<BlogPostRow[]> {
    const supabase = await createServerSupabaseClient();
    const { data } = await supabase
      .from("blog_posts")
      .select("*")
      .order("created_at", { ascending: false });
    return data ?? [];
  }
}

```

```

export async function getPostAdmin(id: string): Promise<BlogPostRow | null> {
  const supabase = await createServerSupabaseClient();
  const { data } = await supabase.from("blog_posts").select("*").eq("id", id).maybeSingle(
    );
  return data ?? null;
}

export async function listOrdersAdmin(): Promise<OrderRow[]> {
  const supabase = await createServerSupabaseClient();
  const { data } = await supabase
    .from("orders")
    .select("*")
    .order("created_at", { ascending: false });
  return data ?? [];
}

export async function adminCounts() {
  const supabase = await createServerSupabaseClient();
  const [products, posts, orders, newOrders] = await Promise.all([
    supabase.from("products").select("id", { count: "exact", head: true }),
    supabase.from("blog_posts").select("id", { count: "exact", head: true }),
    supabase.from("orders").select("id", { count: "exact", head: true }),
    supabase.from("orders").select("id", { count: "exact", head: true }).eq("status", "
      new"),
  ]);
  return {
    products: products.count ?? 0,
    posts: posts.count ?? 0,
    orders: orders.count ?? 0,
    newOrders: newOrders.count ?? 0,
  };
}

```

Step 7: Replace `src/app/admin/(panel)/products/actions.ts`

Whole file. The legacy block at the bottom keeps the old form + list compiling until Tasks 2–3 replace them:

```

"use server";

import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { createServerSupabaseClient } from "@lib/supabase/server";
import {
  validatePublishTier,
  copySlug,
  nextSortOrder,
  sortOrderSequence,
} from "@lib/product-admin";

// -----
// Payload types --the tiered editor submits ONE JSON object (nested lists
// don't fit flat FormData).
// -----

export type EditorFeature = {
  title: string;
  description: string;
  image: string | null;

```

```

};

export type ProductPayload = {
  slug: string;
  name: string;
  tagline: string;
  summary: string;
  highlights: string[];
  description: string;
  technicalDetails: string;
  coverImage: string | null;
  gallery: string[];
  categoryId: string | null;
  tech: string[];
  lifecycle: "live" | "beta" | "soon";
  year: string;
  liveUrl: string;
  repoUrl: string;
  seoDescription: string;
  ogImage: string | null;
  featured: boolean;
  features: EditorFeature[];
  /** Ordered --index becomes the link sort_order. */
  relatedPostIds: string[];
};

export type SaveIntent = "save" | "publish" | "unpublish";

export type SaveProductResult =
  | { ok: true; id: string }
  | { ok: false; missing: string[]; error: string | null };

export type MutationResult = { ok: boolean; error: string | null };

export type UploadResult = { ok: true; url: string } | { ok: false; error: string };

function revalidateProductPaths(slugs: (string | null | undefined)[] = []) {
  revalidatePath("/");
  revalidatePath("/products");
  for (const slug of slugs) if (slug) revalidatePath(`/products/${slug}`);
  revalidatePath("/admin/products");
}

const LIFECYCLES = ["live", "beta", "soon"] as const;

function sanitize(payload: ProductPayload) {
  return {
    slug: payload.slug.trim().toLowerCase(),
    name: payload.name.trim(),
    tagline: payload.tagline.trim() || null,
    summary: payload.summary.trim() || null,
    highlights: payload.highlights.map((h) => h.trim()).filter(Boolean),
    description: payload.description.trim() || null,
    technical_details: payload.technicalDetails.trim() || null,
    cover_image: payload.coverImage || null,
    gallery: payload.gallery.filter(Boolean),
    category_id: payload.categoryId || null,
    tech: payload.tech.map((t) => t.trim()).filter(Boolean),
    lifecycle: LIFECYCLES.includes(payload.lifecycle) ? payload.lifecycle : "live",
    year: payload.year.trim() || null,
  };
}

```

```

    live_url: payload.liveUrl.trim() || null,
    repo_url: payload.repoUrl.trim() || null,
    seo_description: payload.seoDescription.trim() || null,
    og_image: payload.ogImage || null,
    featured: payload.featured,
  };
}

// -----
// saveProductFull --fields + features replace + links replace, one action.
// -----

export async function saveProductFull(
  id: string | null,
  intent: SaveIntent,
  payload: ProductPayload,
): Promise<SaveProductResult> {
  const fields = sanitize(payload);
  if (!fields.name || !fields.slug) {
    return { ok: false, missing: [], error: "Name and slug are required." };
  }

  const supabase = await createServerSupabaseClient();

  let current: { status: "draft" | "published"; slug: string; published_at: string | null } | null =
    null;
  if (id) {
    const { data } = await supabase
      .from("products")
      .select("status, slug, published_at")
      .eq("id", id)
      .maybeSingle();
    if (!data) return { ok: false, missing: [], error: "Product not found." };
    current = data;
  }

  // Spec §3: the Quick View tier must be complete to BE or REMAIN published.
  // "save" on a published row keeps it published, so it re-runs the same
  // validation; the editor offers intent "unpublish" as the escape hatch.
  const targetStatus: "draft" | "published" =
    intent === "publish" ? "published" : intent === "unpublish" ? "draft" : (current?.
      status ?? "draft");

  if (targetStatus === "published") {
    const check = validatePublishTier({
      name: fields.name,
      tagline: fields.tagline ?? "",
      summary: fields.summary ?? "",
      highlights: fields.highlights,
      coverImage: fields.cover_image,
      categoryId: fields.category_id,
    });
    if (!check.valid) return { ok: false, missing: check.missing, error: null };
  }

  const record = {
    ...fields,
    status: targetStatus,
    published_at:

```

```

    targetStatus === "published"
    ? (current?.published_at ?? new Date().toISOString())
    : (current?.published_at ?? null),
  };

let productId: string;
if (id) {
  const { error } = await supabase.from("products").update(record).eq("id", id);
  if (error) return { ok: false, missing: [], error: error.message };
  productId = id;
} else {
  const { data, error } = await supabase.from("products").insert(record).select("id").single();
  if (error || !data) {
    return { ok: false, missing: [], error: error?.message ?? "Insert failed." };
  }
  productId = data.id;
}

// Replace features (delete + insert: tiny volumes, trivial ordering writes).
const delFeatures = await supabase.from("product_features").delete().eq("product_id", productId);
if (delFeatures.error) return { ok: false, missing: [], error: delFeatures.error.message };
const featureRows = payload.features
  .map((f) => ({
    title: f.title.trim(),
    description: f.description.trim() || null,
    image: f.image || null,
  }))
  .filter((f) => f.title)
  .map((f, i) => ({ ...f, product_id: productId, sort_order: (i + 1) * 10 }));
if (featureRows.length > 0) {
  const { error } = await supabase.from("product_features").insert(featureRows);
  if (error) return { ok: false, missing: [], error: error.message };
}

// Replace related-post links.
const delLinks = await supabase.from("product_blog_links").delete().eq("product_id", productId);
if (delLinks.error) return { ok: false, missing: [], error: delLinks.error.message };
if (payload.relatedPostIds.length > 0) {
  const linkRows = payload.relatedPostIds.map((postId, i) => ({
    product_id: productId,
    blog_post_id: postId,
    sort_order: (i + 1) * 10,
  }));
  const { error } = await supabase.from("product_blog_links").insert(linkRows);
  if (error) return { ok: false, missing: [], error: error.message };
}

revalidateProductPaths([fields.slug, current && current.slug !== fields.slug ? current.slug : null]);
return { ok: true, id: productId };
}

// -----
// reorderProducts --persists the admin drag order as sort_order 10, 20, 30 ...
// -----

```

```

export async function reorderProducts(orderedIds: string[]): Promise<MutationResult> {
  const supabase = await createServerSupabaseClient();
  const orders = sortOrderSequence(orderedIds.length);
  for (let i = 0; i < orderedIds.length; i++) {
    const { error } = await supabase
      .from("products")
      .update({ sort_order: orders[i] })
      .eq("id", orderedIds[i]);
    if (error) return { ok: false, error: error.message };
  }
  revalidateProductPaths();
  return { ok: true, error: null };
}

// -----
// duplicateProduct --spec §3: new draft, -copy slug (retry), featured off,
// end of list, features + links copied, image URLs shared by reference.
// -----

export async function duplicateProduct(id: string): Promise<MutationResult> {
  const supabase = await createServerSupabaseClient();

  const { data: source } = await supabase.from("products").select("*").eq("id", id).
    maybeSingle();
  if (!source) return { ok: false, error: "Product not found." };

  const { data: maxRow } = await supabase
    .from("products")
    .select("sort_order")
    .order("sort_order", { ascending: false })
    .limit(1)
    .maybeSingle();

  const copy = {
    name: `${source.name} (copy)`,
    tagline: source.tagline,
    summary: source.summary,
    highlights: source.highlights,
    description: source.description,
    technical_details: source.technical_details,
    cover_image: source.cover_image, // shared Storage object, by reference (spec §3)
    gallery: source.gallery,
    category_id: source.category_id,
    tech: source.tech,
    lifecycle: source.lifecycle,
    year: source.year,
    live_url: source.live_url,
    repo_url: source.repo_url,
    seo_description: source.seo_description,
    og_image: source.og_image,
    featured: false,
    sort_order: nextSortOrder(maxRow ? [maxRow.sort_order] : []),
    status: "draft" as const,
    published_at: null,
  };

  let newId: string | null = null;
  for (let attempt = 1; attempt <= 25 && !newId; attempt++) {
    const { data, error } = await supabase
      .from("products")

```

```

    .insert({ ...copy, slug: copySlug(source.slug, attempt) })
    .select("id")
    .single();
    if (data) newId = data.id;
    else if (error && error.code === "23505") continue; // slug taken --next suffix
    else return { ok: false, error: error?.message ?? "Insert failed." };
  }
  if (!newId) return { ok: false, error: "Could not find a free slug after 25 tries." };
  const newProductId = newId;

  const { data: features } = await supabase
    .from("product_features")
    .select("title,description,image,sort_order")
    .eq("product_id", id);
  if (features && features.length > 0) {
    const { error } = await supabase
      .from("product_features")
      .insert(features.map((f) => ({ ...f, product_id: newProductId })));
    if (error) return { ok: false, error: error.message };
  }

  const { data: links } = await supabase
    .from("product_blog_links")
    .select("blog_post_id,sort_order")
    .eq("product_id", id);
  if (links && links.length > 0) {
    const { error } = await supabase
      .from("product_blog_links")
      .insert(links.map((l) => ({ ...l, product_id: newProductId })));
    if (error) return { ok: false, error: error.message };
  }

  revalidateProductPaths(); // new row is a draft --public unaffected, admin list
    refreshes
  return { ok: true, error: null };
}

// -----
// uploadProductImage --product-images bucket (public read / admin write, 0001).
// Requires the serverActions.bodySizeLimit bump in next.config.ts.
// -----

const MAX_UPLOAD_BYTES = 5 * 1024 * 1024;

export async function uploadProductImage(formData: FormData): Promise<UploadResult> {
  const file = formData.get("file");
  if (!(file instanceof File) || file.size === 0) {
    return { ok: false, error: "Choose a file first." };
  }
  if (!file.type.startsWith("image/")) {
    return { ok: false, error: "Only image files can be uploaded." };
  }
  if (file.size > MAX_UPLOAD_BYTES) {
    return { ok: false, error: "Images must be 5MB or smaller." };
  }

  const ext =
    (file.name.split(".").pop() ?? "png").toLowerCase().replace(/[~a-z0-9]/g, "") || "png";
  const path = `products/${crypto.randomUUID()}.${ext}`;

```



```

const supabase = await createServerSupabaseClient();
const { error } = await supabase.storage
  .from("product-images")
  .upload(path, file, { contentType: file.type, cacheControl: "31536000" });
if (error) return { ok: false, error: error.message };

const { data } = supabase.storage.from("product-images").getPublicUrl(path);
return { ok: true, url: data.publicUrl };
}

// -----
// deleteProduct --unchanged behavior, now also revalidates the detail path.
// -----

export async function deleteProduct(formData: FormData) {
  const id = String(formData.get("id") ?? "");
  const supabase = await createServerSupabaseClient();
  const { data: row } = await supabase.from("products").select("slug").eq("id", id).
    maybeSingle();
  await supabase.from("products").delete().eq("id", id);
  revalidateProductPaths([row?.slug]);
}

// -----
// Legacy form actions --kept so product-form.tsx and the current list page
// compile until their replacements land. parseForm/ProductFormState/create/
// update are removed in Task 2; toggleProductPublished in Task 3 (it bypasses
// publish validation, which spec §3 forbids --the new list has no toggle).
// -----

export type ProductFormState = { error?: string };

function parseForm(formData: FormData) {
  const tech = String(formData.get("tech") ?? "")
    .split(",")
    .map((t) => t.trim())
    .filter(Boolean);
  const lifecycleRaw = String(formData.get("lifecycle") ?? "live").toLowerCase();
  const published = formData.get("published") === "on";
  const existingPublishedAt =
    String(formData.get("existing_published_at") ?? "").trim() || null;
  return {
    slug: String(formData.get("slug") ?? "")
      .trim()
      .toLowerCase(),
    name: String(formData.get("name") ?? "").trim(),
    summary: String(formData.get("summary") ?? "").trim() || null,
    description: String(formData.get("description") ?? "").trim() || null,
    live_url: String(formData.get("live_url") ?? "").trim() || null,
    year: String(formData.get("year") ?? "").trim() || null,
    lifecycle: (["live", "beta", "soon"].includes(lifecycleRaw) ? lifecycleRaw : "live")
      as
      "live" | "beta" | "soon",
    tech,
    featured: formData.get("featured") === "on",
    status: (published ? "published" : "draft") as "draft" | "published",
    published_at: published ? (existingPublishedAt ?? new Date().toISOString()) :
      existingPublishedAt,
  };
}

```

```

}

export async function createProduct(
  _prev: ProductFormState,
  formData: FormData,
): Promise<ProductFormState> {
  const values = parseForm(formData);
  if (!values.name || !values.slug) return { error: "Name and slug are required." };
  const supabase = await createServerSupabaseClient();
  const { error } = await supabase.from("products").insert(values);
  if (error) return { error: error.message };
  revalidateProductPaths([values.slug]);
  redirect("/admin/products");
}

export async function updateProduct(
  id: string,
  _prev: ProductFormState,
  formData: FormData,
): Promise<ProductFormState> {
  const values = parseForm(formData);
  if (!values.name || !values.slug) return { error: "Name and slug are required." };
  const supabase = await createServerSupabaseClient();
  const { error } = await supabase.from("products").update(values).eq("id", id);
  if (error) return { error: error.message };
  revalidateProductPaths([values.slug]);
  redirect("/admin/products");
}

export async function toggleProductPublished(formData: FormData) {
  const id = String(formData.get("id") ?? "");
  const next = formData.get("current") !== "true";
  const supabase = await createServerSupabaseClient();
  const { data: row } = await supabase
    .from("products")
    .select("published_at")
    .eq("id", id)
    .maybeSingle();
  await supabase
    .from("products")
    .update({
      status: next ? "published" : "draft",
      published_at: next ? (row?.published_at ?? new Date().toISOString()) : row?.
        published_at,
    })
    .eq("id", id);
  revalidateProductPaths();
}

```

Step 8: Create `src/app/admin/(panel)/products/categories/actions.ts`

```

"use server";

import { revalidatePath } from "next/cache";
import { createServerSupabaseClient } from "@lib/supabase/server";
import { slugify, nextSortOrder, sortOrderSequence } from "@lib/product-admin";

export type CategoryActionResult = { ok: boolean; error: string | null };

```

```

function revalidateCategoryPaths() {
  revalidatePath("/");
  revalidatePath("/products"); // filter pills come from this table
  revalidatePath("/admin/products");
  revalidatePath("/admin/products/categories");
}

export async function createCategory(name: string): Promise<CategoryActionResult> {
  const trimmed = name.trim();
  if (!trimmed) return { ok: false, error: "Category name is required." };
  const supabase = await createServerSupabaseClient();
  const { data: rows } = await supabase.from("product_categories").select("sort_order");
  const sort_order = nextSortOrder((rows ?? []).map((r) => r.sort_order));
  const { error } = await supabase
    .from("product_categories")
    .insert({ name: trimmed, slug: slugify(trimmed), sort_order });
  if (error) {
    return {
      ok: false,
      error:
        error.code === "23505" ? "A category with that name already exists." : error.
        message,
    };
  }
  revalidateCategoryPaths();
  return { ok: true, error: null };
}

export async function renameCategory(id: string, name: string): Promise<
  CategoryActionResult> {
  const trimmed = name.trim();
  if (!trimmed) return { ok: false, error: "Category name is required." };
  const supabase = await createServerSupabaseClient();
  const { error } = await supabase
    .from("product_categories")
    .update({ name: trimmed, slug: slugify(trimmed) })
    .eq("id", id);
  if (error) {
    return {
      ok: false,
      error:
        error.code === "23505" ? "A category with that name already exists." : error.
        message,
    };
  }
  revalidateCategoryPaths();
  return { ok: true, error: null };
}

/**
 * Spec §3: delete is blocked while products reference the category. The FK is
 * ON DELETE SET NULL (0003), so the database would NOT stop us --this guard is
 * the only thing standing between the admin and silently uncategorized products.
 */
export async function deleteCategory(id: string): Promise<CategoryActionResult> {
  const supabase = await createServerSupabaseClient();
  const { count } = await supabase
    .from("products")
    .select("id", { count: "exact", head: true })

```

```

    .eq("category_id", id);
    if (count && count > 0) {
      return {
        ok: false,
        error: `${count} product${count === 1 ? "" : "s"} still use${count === 1 ? "s" : ""} this category --reassign ${count === 1 ? "it" : "them"} in the product editor first.`,
      };
    }
  }
  const { error } = await supabase.from("product_categories").delete().eq("id", id);
  if (error) return { ok: false, error: error.message };
  revalidateCategoryPaths();
  return { ok: true, error: null };
}

export async function reorderCategories(orderedIds: string[]): Promise<
  CategoryActionResult> {
  const supabase = await createServerSupabaseClient();
  const orders = sortOrderSequence(orderedIds.length);
  for (let i = 0; i < orderedIds.length; i++) {
    const { error } = await supabase
      .from("product_categories")
      .update({ sort_order: orders[i] })
      .eq("id", orderedIds[i]);
    if (error) return { ok: false, error: error.message };
  }
  revalidateCategoryPaths();
  return { ok: true, error: null };
}

```

Step 9: Bump the server-action body limit

Replace `next.config.ts` with:

```

import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  // uploadProductImage accepts files up to 5MB; Next's server-action default
  // is 1MB. 6mb leaves headroom for multipart overhead.
  experimental: {
    serverActions: { bodySizeLimit: "6mb" },
  },
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "axbsghyqhhdaiylcksbv.supabase.co",
        pathname: "/storage/v1/object/public/**",
      },
    ],
  },
};

export default nextConfig;

```

If `next` build in Step 10 warns that `experimental.serverActions` is unrecognized (Next 16 may have promoted it), move the `serverActions` block to the config top level and re-run — the green bar decides.

Step 10: Green bar

Run: `npm run typecheck && npm run lint && npm test && npm run build` → all green (build proves the old list/edit pages still compile against the legacy exports).

Step 11: Commit

```
git add src/lib/product-admin.ts src/lib/product-admin.test.ts src/lib/admin-queries.ts
      "src/app/admin/(panel)/products/actions.ts" "src/app/admin/(panel)/products/
      categories/actions.ts" next.config.ts
git commit -m "feat: products admin data layer --publish validation, slug/sort helpers,
      full-save/reorder/duplicate/upload/category actions"
```

6 Task 2: Tiered editor UI (Quick View / Features / Technical / SEO & Links)

Files

- Modify: `package.json` (via `npm install` — the one sanctioned dependency addition)
- Create: `src/components/admin/form-styles.ts`
- Create: `src/components/admin/sortable-list.tsx`
- Create: `src/components/admin/image-upload.tsx`
- Create: `src/components/admin/related-posts-picker.tsx`
- Create: `src/components/admin/product-editor.tsx`, `src/components/admin/product-editor.test.ts`
- Modify: `src/app/admin/(panel)/products/new/page.tsx`, `src/app/admin/(panel)/products/[id]/page.tsx` (whole-file replaces)
- Delete: `src/components/admin/product-form.tsx`
- Modify: `src/app/admin/(panel)/products/actions.ts` (remove legacy block A), `src/lib/admin-queries.ts` (remove `getProductAdmin`)

Interfaces

- Consumes: everything Task 1 produced; `ProductCategoryRow` from `types`; `buttonVariants`, `cn`.
- Produces: `<ProductEditor product={AdminProductFull | null} categories={ProductCategoryRow[]} posts={PostPickerItem[]} />`; `<SortableList ids onReorder disabled?> + <SortableItem id disabled? handleTitle? className?>` (Task 3 reuses both); `<ImageUploadField label destination value onChange />`; `<GalleryUploadField value onChange />`; `<RelatedPostsPicker posts selectedIds onChange />`; `fieldClass/labelClass/hintClass` from `form-styles`.

Step 1: Install @dnd-kit

THE one new dependency of this feature, spec §6:

```
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
```

Expected: `package.json` dependencies now include `@dnd-kit/core`, `@dnd-kit/sortable`, `@dnd-kit/utilities`; `lockfile` updated; no other dependency changes.

Step 2: Create `src/components/admin/form-styles.ts`

The strings previously private to `product-form.tsx`, now shared by `editor/picker/table/manager`:

```
export const fieldClass =
  "w-full rounded-lg border border-input bg-background px-3.5 py-2.5 text-sm text-
    foreground outline-none transition-colors placeholder:text-muted-foreground focus-
    visible:border-ring focus-visible:ring-3 focus-visible:ring-ring/40";

export const labelClass = "flex flex-col gap-1.5 text-sm font-medium";

/** For the "(public destination)" hint next to every field label. */
export const hintClass = "text-muted-foreground font-normal";
```

Step 3: Create `src/components/admin/sortable-list.tsx`

```
"use client";

import {
  DndContext,
  KeyboardSensor,
  PointerSensor,
  closestCenter,
  useSensor,
  useSensors,
  type DragEndEvent,
} from "@dnd-kit/core";
import {
  SortableContext,
  arrayMove,
  sortableKeyboardCoordinates,
  useSortable,
  verticalListSortingStrategy,
} from "@dnd-kit/sortable";
import { CSS } from "@dnd-kit/utilities";
import { GripVertical } from "lucide-react";
import { cn } from "@lib/utills";

/**
 * Vertical drag-to-reorder primitives (@dnd-kit: keyboard + touch, spec §6).
 * SortableList owns the DndContext; render SortableItem rows inside in the
 * same order as 'ids'. onReorder receives the full reordered id array.
 * Shared by: highlights, features, related posts, product list, categories.
 */
export function SortableList({
  ids,
  onReorder,
  disabled = false,
  children,
}: {
  ids: string[];
  onReorder: (ids: string[]) => void;
  disabled?: boolean;
  children: React.ReactNode;
}) {
  const sensors = useSensors(
    useSensor(PointerSensor, { activationConstraint: { distance: 4 } }),
    useSensor(KeyboardSensor, { coordinateGetter: sortableKeyboardCoordinates })
  );
```

```

function handleDragEnd(event: DragEndEvent) {
  const { active, over } = event;
  if (!over || active.id === over.id) return;
  const from = ids.indexOf(String(active.id));
  const to = ids.indexOf(String(over.id));
  if (from === -1 || to === -1) return;
  onReorder(arrayMove(ids, from, to));
}

return (
  <DndContext sensors={sensors} collisionDetection={closestCenter} onDragEnd={
    handleDragEnd}>
    <SortableContext items={ids} strategy={verticalListSortingStrategy} disabled={
      disabled}>
      {children}
    </SortableContext>
  </DndContext>
);
}

export function SortableItem({
  id,
  disabled = false,
  handleTitle,
  className,
  children,
}: {
  id: string;
  disabled?: boolean;
  /** Tooltip on the handle --used to say WHY dragging is off (filters active). */
  handleTitle?: string;
  className?: string;
  children: React.ReactNode;
}) {
  const { attributes, listeners, setNodeRef, transform, transition, isDragging } =
    useSortable({
      id,
      disabled,
    });
  return (
    <div
      ref={setNodeRef}
      style={{ transform: CSS.Transform.toString(transform), transition }}
      className={cn("flex items-start gap-2", isDragging && "z-10 opacity-70", className)}
    >
      <button
        type="button"
        title={handleTitle ?? "Drag to reorder"}
        aria-label={handleTitle ?? "Drag to reorder"}
        disabled={disabled}
        {...attributes}
        {...listeners}
        className={cn(
          "text-muted-foreground hover:text-foreground mt-2 shrink-0 touch-none rounded p-1 transition-colors",
          disabled ? "cursor-not-allowed opacity-40" : "cursor-grab active:cursor-grabbing",
        )}
      >

```

```

    })
  >
    <GripVertical className="size-4" />
  </button>
  <div className="min-w-0 flex-1">{children}</div>
</div>
);
}

```

Step 4: Create `src/components/admin/image-upload.tsx`

```

"use client";

import { useRef, useState } from "react";
import Image from "next/image";
import { ImagePlus, Loader2, X } from "lucide-react";
import { uploadProductImage } from "@app/admin/(panel)/products/actions";

/** Shared pick-file →server-action-upload flow. */
function useUpload(onDone: (url: string) => void) {
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function upload(file: File) {
    setBusy(true);
    setError(null);
    const formData = new FormData();
    formData.set("file", file);
    const result = await uploadProductImage(formData);
    if (result.ok) onDone(result.url);
    else setError(result.error);
    setBusy(false);
  }

  return { busy, error, upload };
}

export function ImageUploadField({
  label,
  destination,
  value,
  onChange,
}: {
  label: string;
  /** Where this image appears publicly --every admin field names its destination. */
  destination: string;
  value: string | null;
  onChange: (url: string | null) => void;
}) {
  const inputRef = useRef<HTMLInputElement>(null);
  const { busy, error, upload } = useUpload((url) => onChange(url));

  return (
    <div className="flex flex-col gap-1.5 text-sm font-medium">
      <span>
        {label} <span className="text-muted-foreground font-normal">({destination})</span>
      </span>
      <input
        ref={inputRef}
        type="text"
        value={value}
        onChange={onChange}
      />
    </div>
  );
}

```



```

<div className="flex items-start gap-3">
  <div className="border-border bg-muted relative aspect-[16/9] w-40 shrink-0
    overflow-hidden rounded-lg border">
    {value ? (
      <Image src={value} alt="" fill sizes="10rem" className="object-cover" />
    ) : (
      <span className="text-muted-foreground absolute inset-0 grid place-items-
        center text-xs font-normal">
        No image
      </span>
    )}
  </div>
  <div className="flex flex-col gap-2">
    <input
      ref={inputRef}
      type="file"
      accept="image/*"
      className="hidden"
      onChange={(e) => {
        const file = e.target.files?.[0];
        if (file) void upload(file);
        e.target.value = "";
      }}
    />
    <button
      type="button"
      disabled={busy}
      onClick={() => inputRef.current?.click()}
      className="border-border text-muted-foreground hover:text-foreground inline-
        flex w-fit items-center gap-1.5 rounded-md border px-3 py-1.5 text-xs font-
        -medium transition-colors disabled:opacity-50"
    >
      {busy ? (
        <Loader2 className="size-3.5 animate-spin" />
      ) : (
        <ImagePlus className="size-3.5" />
      )}
      {busy ? "Uploading..." : value ? "Replace image" : "Upload image"}
    </button>
    {value ? (
      <button
        type="button"
        onClick={() => onChange(null)}
        className="text-muted-foreground hover:text-destructive inline-flex w-fit
          items-center gap-1 text-xs transition-colors"
      >
        <X className="size-3" />
        Remove
      </button>
    ) : null}
    {error ? <p className="text-destructive text-xs font-normal">{error}</p> : null
    }
  </div>
</div>
</div>
</div>
);
}

export function GalleryUploadField({
  value,

```

```

    onChange,
  }: {
    value: string[];
    onChange: (urls: string[]) => void;
  }) {
    const inputRef = useRef<HTMLInputElement>(null);
    const { busy, error, upload } = useUpload((url) => onChange([...value, url]));

    return (
      <div className="flex flex-col gap-1.5 text-sm font-medium">
        <span>
          Gallery{" "}
          <span className="text-muted-foreground font-normal">
            (image grid on the technical page)
          </span>
        </span>
        <div className="flex flex-wrap gap-3">
          {value.map((url) => (
            <div
              key={url}
              className="border-border bg-muted relative aspect-[16/10] w-36 overflow-hidden rounded-lg border"
            >
              <Image src={url} alt="" fill sizes="9rem" className="object-cover" />
              <button
                type="button"
                aria-label="Remove gallery image"
                onClick={() => onChange(value.filter((u) => u !== url))}
                className="bg-background/80 text-foreground hover:text-destructive absolute top-1 right-1 rounded-full p-1 backdrop-blur transition-colors"
              >
                <X className="size-3" />
              </button>
            </div>
          ))}
          <input
            ref={inputRef}
            type="file"
            accept="image/*"
            className="hidden"
            onChange={(e) => {
              const file = e.target.files?.[0];
              if (file) void upload(file);
              e.target.value = "";
            }}
          />
          <button
            type="button"
            disabled={busy}
            onClick={() => inputRef.current?.click()}
            className="border-border text-muted-foreground hover:text-foreground grid aspect-[16/10] w-36 place-items-center rounded-lg border border-dashed text-xs font-medium transition-colors disabled:opacity-50"
          >
            {busy ? <Loader2 className="size-4 animate-spin" /> : "+ Add image"}
          </button>
        </div>
        {error ? <p className="text-destructive text-xs font-normal">{error}</p> : null}
      </div>
    );
  }

```

```
}

```

Step 5: Create src/components/admin/related-posts-picker.tsx

```
"use client";

import { useMemo, useState } from "react";
import { X } from "lucide-react";
import type { PostPickerItem } from "@lib/admin-queries";
import { SortableList, SortableItem } from "@components/admin/sortable-list";
import { fieldClass } from "@components/admin/form-styles";

/**
 * Spec §3 SEO & Links tab: search posts by title, multi-select, drag-order.
 * Drafts are selectable but badged --anon RLS hides them publicly until the
 * post is published, so the link "activates" on publish with no extra work.
 */
export function RelatedPostsPicker({
  posts,
  selectedIds,
  onChange,
}: {
  posts: PostPickerItem[];
  selectedIds: string[];
  onChange: (ids: string[]) => void;
}) {
  const [query, setQuery] = useState("");
  const byId = useMemo(() => new Map(posts.map((p) => [p.id, p])), [posts]);

  const results = useMemo(() => {
    const q = query.trim().toLowerCase();
    return posts.filter(
      (p) => !selectedIds.includes(p.id) && (q === "" || p.title.toLowerCase().includes(q)),
    );
  }, [posts, query, selectedIds]);

  return (
    <div className="flex flex-col gap-3 text-sm font-medium">
      <span>
        Related blog posts{" "}
        <span className="text-muted-foreground font-normal">
          (chips in the quick view “Related writing” cards on the technical page)
        </span>
      </span>

      {selectedIds.length > 0 ? (
        <SortableList ids={selectedIds} onReorder={onChange}>
          <div className="flex flex-col gap-2">
            {selectedIds.map((id) => {
              const post = byId.get(id);
              if (!post) return null;
              return (
                <SortableItem
                  key={id}
                  id={id}
                  className="border-border bg-card items-center rounded-lg border p-2"
                >

```

```

    <div className="flex items-center justify-between gap-2">
      <span className="truncate text-sm">{post.title}</span>
      <span className="flex shrink-0 items-center gap-2">
        {post.status === "draft" ? (
          <span className="bg-amber-500/12 rounded-full px-2 py-0.5 text-[11px] font-medium text-amber-500 ring-1 ring-amber-500/20 ring-inset">
            Draft --won't show until published
          </span>
        ) : null}
      <button
        type="button"
        aria-label={`Remove ${post.title}`}
        onClick={() => onChange(selectedIds.filter((s) => s !== id))}
        className="text-muted-foreground hover:text-destructive p-1 transition-colors"
      >
        <X className="size-3.5" />
      </button>
    </span>
  </div>
</SortableItem>
);
}}
</div>
</SortableList>
) : (
  <p className="text-muted-foreground text-xs font-normal">No posts linked yet.</p>
)

<input
  value={query}
  onChange={(e) => setQuery(e.target.value)}
  placeholder="Search posts by title..."
  aria-label="Search posts by title"
  className={fieldClass}
/>
<div className="border-border max-h-56 overflow-y-auto rounded-lg border">
  {results.length === 0 ? (
    <p className="text-muted-foreground p-3 text-xs font-normal">No matching posts
    .</p>
  ) : (
    results.map((post) => (
      <button
        key={post.id}
        type="button"
        onClick={() => onChange([...selectedIds, post.id])}
        className="hover:bg-muted flex w-full items-center justify-between gap-2 px-3 py-2 text-left text-sm font-normal transition-colors"
      >
        <span className="truncate">{post.title}</span>
        {post.status === "draft" ? (
          <span className="text-muted-foreground shrink-0 text-[11px]">draft</span>
        ) : null}
      </button>
    ))
  )}
</div>
</div>
);

```

```
}

```

Step 6: Create `src/components/admin/product-editor.tsx`

The tiered editor. One client island; all state lifted here (tabs render conditionally without losing anything); submits the Task 1 action with a JSON payload:

```
"use client";

import { useMemo, useState } from "react";
import Link from "next/link";
import { useRouter } from "next/navigation";
import { Eye, Plus, X } from "lucide-react";
import type { ProductCategoryRow } from "@lib/supabase/types";
import type { AdminProductFull, PostPickerItem } from "@lib/admin-queries";
import {
  saveProductFull,
  type ProductPayload,
  type SaveIntent,
} from "@app/admin/(panel)/products/actions";
import { validatePublishTier, slugify } from "@lib/product-admin";
import { SortableList, SortableItem } from "@components/admin/sortable-list";
import { ImageUploadField, GalleryUploadField } from "@components/admin/image-upload";
import { RelatedPostsPicker } from "@components/admin/related-posts-picker";
import { fieldClass, labelClass, hintClass } from "@components/admin/form-styles";
import { buttonVariants } from "@components/ui/button";
import { cn } from "@lib/utils";

/** Client-side rows need stable keys for drag-and-drop. */
type HighlightItem = { key: string; text: string };
type FeatureItem = { key: string; title: string; description: string; image: string | null };

type Tab = "quick" | "features" | "technical" | "seo";

const TABS: { id: Tab; label: string; hint: string }[] = [
  { id: "quick", label: "Quick View", hint: "the box everyone sees first" },
  { id: "features", label: "Features", hint: "the detail page's showcase" },
  { id: "technical", label: "Technical", hint: "the detail page" },
  { id: "seo", label: "SEO & Links", hint: "search results & related posts" },
];

function newKey() {
  return typeof crypto !== "undefined" && "randomUUID" in crypto
    ? crypto.randomUUID()
    : Math.random().toString(36).slice(2);
}

export function ProductEditor({
  product,
  categories,
  posts,
}: {
  product: AdminProductFull | null;
  categories: ProductCategoryRow[];
  posts: PostPickerItem[];
}) {
  const router = useRouter();
  const isPublished = product?.status === "published";

```

```

const [tab, setTab] = useState<Tab>("quick");
const [busy, setBusy] = useState<SaveIntent | null>(null);
const [missing, setMissing] = useState<string[]>([]);
const [serverError, setServerError] = useState<string | null>(null);
/** True after a blocked save on a published product --shows the escape hatch. */
const [offerUnpublish, setOfferUnpublish] = useState(false);

// ---- the form object (keyed lists for dnd) ----
const [name, setName] = useState(product?.name ?? "");
const [slug, setSlug] = useState(product?.slug ?? "");
const [slugTouched, setSlugTouched] = useState(Boolean(product));
const [tagline, setTagline] = useState(product?.tagline ?? "");
const [summary, setSummary] = useState(product?.summary ?? "");
const [highlights, setHighlights] = useState<HighlightItem[]>(
  (product?.highlights ?? []).map((text) => ({ key: newKey(), text })),
);
const [categoryId, setCategoryId] = useState(product?.category_id ?? "");
const [coverImage, setCoverImage] = useState<string | null>(product?.cover_image ??
  null);
const [liveUrl, setLiveUrl] = useState(product?.live_url ?? "");
const [lifecycle, setLifecycle] = useState<"live" | "beta" | "soon">(
  product?.lifecycle ?? "live",
);
const [featured, setFeatured] = useState(product?.featured ?? false);
const [features, setFeatures] = useState<FeatureItem[]>(
  [...(product?.product_features ?? [])]
    .sort((a, b) => a.sort_order - b.sort_order)
    .map((f) => ({
      key: newKey(),
      title: f.title,
      description: f.description ?? "",
      image: f.image,
    })),
);
const [description, setDescription] = useState(product?.description ?? "");
const [technicalDetails, setTechnicalDetails] = useState(product?.technical_details ??
  "");
const [tech, setTech] = useState((product?.tech ?? []).join(", "));
const [gallery, setGallery] = useState<string[]>(product?.gallery ?? []);
const [repoUrl, setRepoUrl] = useState(product?.repo_url ?? "");
const [year, setYear] = useState(product?.year ?? "");
const [seoDescription, setSeoDescription] = useState(product?.seo_description ?? "");
const [ogImage, setOgImage] = useState<string | null>(product?.og_image ?? null);
const [relatedPostIds, setRelatedPostIds] = useState<string[]>(
  [...(product?.product_blog_links ?? [])]
    .sort((a, b) => a.sort_order - b.sort_order)
    .map((l) => l.blog_post_id),
);

// Live quick-view completeness --drives the Publish button + inline checklist.
const publishCheck = useMemo(
  () =>
    validatePublishTier({
      name,
      tagline,
      summary,
      highlights: highlights.map((h) => h.text),
      coverImage,
      categoryId: categoryId || null,

```

```

    }),
    [name, tagline, summary, highlights, coverImage, categoryId],
  );

function buildPayload(): ProductPayload {
  return {
    slug,
    name,
    tagline,
    summary,
    highlights: highlights.map((h) => h.text),
    description,
    technicalDetails,
    coverImage,
    gallery,
    categoryId: categoryId || null,
    tech: tech.split(",").map((t) => t.trim()).filter(Boolean),
    lifecycle,
    year,
    liveUrl,
    repoUrl,
    seoDescription,
    ogImage,
    featured,
    features: features.map((f) => ({
      title: f.title,
      description: f.description,
      image: f.image,
    })),
    relatedPostIds,
  };
}

async function submit(intent: SaveIntent) {
  setBusy(intent);
  setMissing([]);
  setServerError(null);
  setOfferUnpublish(false);
  const result = await saveProductFull(product?.id ?? null, intent, buildPayload());
  if (!result.ok) {
    setMissing(result.missing);
    setServerError(result.error);
    if (result.missing.length > 0 && isPublished && intent === "save")
      setOfferUnpublish(true);
    setBusy(null);
    return;
  }
  if (product) {
    router.push("/admin/products");
  } else {
    // New product: land in its editor so Preview / Publish are one click away.
    router.push(`/admin/products/${result.id}`);
  }
  router.refresh();
}

return (
  <div className="flex max-w-3xl flex-col gap-6">
    {/* Error banner --names the missing quick-view fields (spec §3). */}
    {missing.length > 0 || serverError ? (

```

```

<div className="border-destructive/30 bg-destructive/10 text-destructive flex
  flex-col gap-2 rounded-lg border px-4 py-3 text-sm">
  {missing.length > 0 ? (
    <>
      <p className="font-medium">
        {isPublished
          ? "A published product can't be saved with an incomplete Quick View.
            Missing:"
          : "The Quick View tab must be complete before publishing. Missing:"}
      </p>
      <ul className="list-inside list-disc">
        {missing.map((m) => (
          <li key={m}>{m}</li>
        ))}
      </ul>
      {offerUnpublish ? (
        <button
          type="button"
          disabled={busy !== null}
          onClick={() => void submit("unpublish")}
          className="border-destructive/40 hover:bg-destructive/10 mt-1 w-fit
            rounded-md border px-3 py-1.5 text-xs font-medium transition-colors"
        >
          Unpublish & save as draft instead
        </button>
      ) : null}
    </>
  ) : null}
  {serverError ? <p>{serverError}</p> : null}
</div>
) : null}

{/* Tier tabs --mirror the public surfaces (spec §3). */}
<div
  role="tablist"
  aria-label="Editor sections"
  className="border-border flex flex-wrap gap-1 border-b"
>
  {TABS.map((t) => (
    <button
      key={t.id}
      type="button"
      role="tab"
      aria-selected={tab === t.id}
      onClick={() => setTab(t.id)}
      className={cn(
        "-mb-px rounded-t-lg border-b-2 px-4 py-2 text-sm font-medium transition-
          colors",
        tab === t.id
          ? "border-primary text-foreground"
          : "text-muted-foreground hover:text-foreground border-transparent",
      )}
    >
      {t.label}
      <span className="text-muted-foreground ml-1.5 hidden text-xs font-normal sm:
        inline">
        --{t.hint}
      </span>
    </button>
  ))}

```



```

</div>

{/* ===== Tab 1: Quick View ===== */}
{tab === "quick" ? (
  <div className="flex flex-col gap-5">
    <div className="grid gap-5 sm:grid-cols-2">
      <label className={labelClass}>
        Name <span className={hintClass}>(card & page title)</span>
        <input
          value={name}
          onChange={(e) => {
            setName(e.target.value);
            if (!slugTouched) setSlug(slugify(e.target.value));
          }}
          className={fieldClass}
        />
      </label>
      <label className={labelClass}>
        Slug <span className={hintClass}>(/products/... --auto from name, editable)
        </span>
        <input
          value={slug}
          onChange={(e) => {
            setSlugTouched(true);
            setSlug(e.target.value);
          }}
          placeholder="lowercase-with-dashes"
          className={fieldClass}
        />
      </label>
    </div>

    <label className={labelClass}>
      Tagline <span className={hintClass}>(card + quick-view headline)</span>
      <input
        value={tagline}
        onChange={(e) => setTagline(e.target.value)}
        className={fieldClass}
      />
    </label>

    <label className={labelClass}>
      Summary <span className={hintClass}>(quick-view intro)</span>
      <input
        value={summary}
        onChange={(e) => setSummary(e.target.value)}
        className={fieldClass}
      />
    </label>

    <div className="flex flex-col gap-1.5 text-sm font-medium">
      <span>
        Highlights{" "}
        <span className={hintClass}>
          (quick-view benefit bullets --plain language, no jargon; 3-6)
        </span>
      </span>
      <SortableList
        ids={highlights.map((h) => h.key)}
        onReorder={(keys) =>

```

```

        setHighlights(
            keys.flatMap((k) => highlights.filter((h) => h.key === k)),
        )
    }
}
>
<div className="flex flex-col gap-2">
    {highlights.map((h) => (
        <SortableItem key={h.key} id={h.key} className="items-center">
            <div className="flex items-center gap-2">
                <input
                    value={h.text}
                    onChange={(e) =>
                        setHighlights(
                            highlights.map((x) =>
                                x.key === h.key ? { ...x, text: e.target.value } : x,
                            ),
                        )
                    }
                    placeholder="One clear benefit"
                    aria-label="Highlight"
                    className={fieldClass}
                />
                <button
                    type="button"
                    aria-label="Remove highlight"
                    onClick={() => setHighlights(highlights.filter((x) => x.key !== h.key))}
                    className="text-muted-foreground hover:text-destructive p-1.5 transition-colors"
                >
                    <X className="size-4" />
                </button>
            </div>
        </SortableItem>
    ))}
</div>
</SortableList>
<button
    type="button"
    onClick={() => setHighlights([...highlights, { key: newKey(), text: "" }])}
    className="text-muted-foreground hover:text-foreground inline-flex w-fit items-center gap-1.5 text-xs font-medium transition-colors"
>
    <Plus className="size-3.5" />
    Add highlight
</button>
</div>

<div className="grid gap-5 sm:grid-cols-2">
    <div className="flex flex-col gap-1.5 text-sm font-medium">
        <span>
            Category <span className={hintClass}>(card pill + grid filter)</span>
        </span>
        <select
            value={categoryId}
            onChange={(e) => setCategoryId(e.target.value)}
            aria-label="Category"
            className={fieldClass}
        >
            <option value="">-- None --</option>

```

```

        {categories.map((c) => (
            <option key={c.id} value={c.id}>
                {c.name}
            </option>
        ))}
    </select>
    <Link
        href="/admin/products/categories"
        className="text-primary w-fit text-xs font-normal hover:underline"
    >
        Manage categories
    </Link>
</div>
<label className={labelClass}>
    Lifecycle <span className={hintClass}>(badge: Live / Beta / Soon)</span>
    <select
        value={lifecycle}
        onChange={(e) => setLifecycle(e.target.value as "live" | "beta" | "soon")}
        className={fieldClass}
    >
        <option value="live">Live</option>
        <option value="beta">Beta</option>
        <option value="soon">Soon</option>
    </select>
</label>
</div>

<ImageUploadField
    label="Cover image"
    destination="card, quick view & page header"
    value={coverImage}
    onChange={setCoverImage}
/>

<div className="grid gap-5 sm:grid-cols-2">
    <label className={labelClass}>
        Live URL{" "}
        <span className={hintClass}>("Open live app" button --empty = Coming soon)</span>
        <input
            value={liveUrl}
            onChange={(e) => setLiveUrl(e.target.value)}
            className={fieldClass}
        />
    </label>
    <label className="flex items-center gap-2 pt-6 text-sm font-medium">
        <input
            type="checkbox"
            checked={featured}
            onChange={(e) => setFeatured(e.target.checked)}
            className="size-4"
        />
        Featured <span className={hintClass}>(pinned first on the grid)</span>
    </label>
</div>
</div>
) : null}

{/* ===== Tab 2: Features ===== */}
{tab === "features" ? (

```

```

<div className="flex flex-col gap-4">
  <p className="text-muted-foreground text-sm">
    Picture + explanation blocks on the technical page, in this order. Optional --
    the
    public section hides itself when there are none.
  </p>
  <SortableList
    ids={features.map((f) => f.key)}
    onReorder={({keys) => setFeatures(keys.flatMap((k) => features.filter((f) => f.
      key === k)))}
  >
    <div className="flex flex-col gap-4">
      {features.map((f) => (
        <SortableItem
          key={f.key}
          id={f.key}
          className="border-border bg-card rounded-xl border p-4"
        >
          <div className="flex flex-col gap-3">
            <div className="flex items-center justify-between">
              <span className="text-muted-foreground text-xs font-medium uppercase">
                Feature
              </span>
              <button
                type="button"
                aria-label="Remove feature"
                onClick={() => setFeatures(features.filter((x) => x.key !== f.key))}
              >
                <X className="size-4" />
              </button>
            </div>
            <label className={labelClass}>
              Title
              <input
                value={f.title}
                onChange={(e) =>
                  setFeatures(
                    features.map((x) =>
                      x.key === f.key ? { ...x, title: e.target.value } : x,
                    ),
                  )
                >
              </label>
              <label className={labelClass}>
                Explanation <span className={hintClass}>(shown beside the picture)</span>
                <textarea
                  rows={3}
                  value={f.description}
                  onChange={(e) =>
                    setFeatures(
                      features.map((x) =>
                        x.key === f.key ? { ...x, description: e.target.value } : x,
                      ),

```

```

        )
      }
      className={cn(fieldClass, "resize-y")}
    />
  </label>
  <ImageUploadField
    label="Image"
    destination="the picture in this block"
    value={f.image}
    onChange={(url) =>
      setFeatures(
        features.map((x) => (x.key === f.key ? { ...x, image: url } : x))
      )
    }
  />
</div>
</SortableItem>
  )]}
</div>
</SortableList>
<button
  type="button"
  onClick={() =>
    setFeatures([
      ...features,
      { key: newKey(), title: "", description: "", image: null },
    ])
  }
  className="text-muted-foreground hover:text-foreground inline-flex w-fit items
    -center gap-1.5 text-xs font-medium transition-colors"
>
  <Plus className="size-3.5" />
  Add feature
</button>
</div>
) : null}

{/* ===== Tab 3: Technical ===== */}
{tab === "technical" ? (
  <div className="flex flex-col gap-5">
    <label className={labelClass}>
      Long description{" "}
      <span className={hintClass}>(technical page body --blank line between
        paragraphs)</span>
    <textarea
      rows={6}
      value={description}
      onChange={(e) => setDescription(e.target.value)}
      className={cn(fieldClass, "resize-y")}
    />
  </label>
  <label className={labelClass}>
    Technical details{" "}
    <span className={hintClass}>
      (“Technical details” section --architecture, stack decisions, notes)
    </span>
    <textarea
      rows={6}
      value={technicalDetails}

```

```

        onChange={(e) => setTechnicalDetails(e.target.value)}
        className={cn(fieldClass, "resize-y")}
      />
    </label>
    <div className="grid gap-5 sm:grid-cols-2">
      <label className={labelClass}>
        Tech <span className={hintClass}>(chips on the technical page --comma
          separated)</span>
        <input
          value={tech}
          onChange={(e) => setTech(e.target.value)}
          placeholder="Next.js, Supabase"
          className={fieldClass}
        />
      </label>
      <label className={labelClass}>
        Year <span className={hintClass}>(header meta line)</span>
        <input value={year} onChange={(e) => setYear(e.target.value)} className={
          fieldClass} />
      </label>
    </div>
    <GalleryUploadField value={gallery} onChange={setGallery} />
    <label className={labelClass}>
      Repo URL <span className={hintClass}>("View repo" button --optional)</span>
      <input
        value={repoUrl}
        onChange={(e) => setRepoUrl(e.target.value)}
        className={fieldClass}
      />
    </label>
  </div>
) : null}

{/* ===== Tab 4: SEO & Links ===== */}
{tab === "seo" ? (
  <div className="flex flex-col gap-5">
    <label className={labelClass}>
      SEO description{" "}
      <span className={hintClass}>(search results --falls back to tagline + summary)
    </span>
    <textarea
      rows={3}
      value={seoDescription}
      onChange={(e) => setSeoDescription(e.target.value)}
      className={cn(fieldClass, "resize-y")}
    />
  </label>
  <ImageUploadField
    label="OG image"
    destination="social shares --falls back to the cover"
    value={ogImage}
    onChange={setOgImage}
  />
  <RelatedPostsPicker
    posts={posts}
    selectedIds={relatedPostIds}
    onChange={setRelatedPostIds}
  />
</div>
) : null}

```

```

{ /* ===== Save row ===== */}
<div className="border-border flex flex-wrap items-center gap-3 border-t pt-5">
  {isPublished ? (
    <>
      <button
        type="button"
        disabled={busy !== null}
        onClick={() => void submit("save")}
        className={cn(buttonVariants({ size: "lg" })))}
      >
        {busy === "save" ? "Saving..." : "Save changes"}
      </button>
      <button
        type="button"
        disabled={busy !== null}
        onClick={() => void submit("unpublish")}
        className={cn(buttonVariants({ variant: "outline", size: "lg" })))}
      >
        {busy === "unpublish" ? "Saving..." : "Unpublish & save as draft"}
      </button>
    </>
  ) : (
    <>
      <button
        type="button"
        disabled={busy !== null}
        onClick={() => void submit("save")}
        className={cn(buttonVariants({ variant: "outline", size: "lg" })))}
      >
        {busy === "save" ? "Saving..." : "Save draft"}
      </button>
      <button
        type="button"
        disabled={busy !== null || !publishCheck.valid}
        title={
          publishCheck.valid
            ? undefined
            : 'Complete the Quick View tab first: ${publishCheck.missing.join(", ")}'
        }
        onClick={() => void submit("publish")}
        className={cn(buttonVariants({ size: "lg" }), !publishCheck.valid && "
          opacity-50")}
      >
        {busy === "publish" ? "Publishing..." : "Publish"}
      </button>
    </>
  )}
  {product ? (
    <Link
      href={`'/admin/products/${product.id}/preview`}
      className={cn(buttonVariants({ variant: "outline", size: "lg" }), "gap-1.5")}
    >
      <Eye className="size-4" />
      Preview
    </Link>
  ) : null}
  <Link
    href="/admin/products"

```

```

        className={buttonVariants({ variant: "outline", size: "lg" })}
      >
        Cancel
      </Link>
      {!publishCheck.valid && !isPublished ? (
        <p className="text-muted-foreground w-full text-xs">
          To publish, complete: {publishCheck.missing.join(" .")}
        </p>
      ) : null}
    </div>
  </div>
);
}

```

Step 7: Rewrite the new/edit pages

Replace `src/app/admin/(panel)/products/new/page.tsx` with:

```

import Link from "next/link";
import { ArrowLeft } from "lucide-react";
import { ProductEditor } from "@components/admin/product-editor";
import { listCategoriesAdmin, listPostsForPicker } from "@lib/admin-queries";

export const dynamic = "force-dynamic";

export default async function NewProductPage() {
  const [categories, posts] = await Promise.all([listCategoriesAdmin(),
    listPostsForPicker()]);

  return (
    <div className="flex flex-col gap-6">
      <Link
        href="/admin/products"
        className="text-muted-foreground hover:text-foreground inline-flex items-center
          gap-1.5 text-sm transition-colors"
      >
        <ArrowLeft className="size-4" />
        Products
      </Link>
      <h1 className="font-heading text-2xl font-bold tracking-tight">New product</h1>
      <ProductEditor product={null} categories={categories} posts={posts} />
    </div>
  );
}

```

Replace `src/app/admin/(panel)/products/[id]/page.tsx` with:

```

import Link from "next/link";
import { notFound } from "next/navigation";
import { ArrowLeft } from "lucide-react";
import {
  getProductFullAdmin,
  listCategoriesAdmin,
  listPostsForPicker,
} from "@lib/admin-queries";
import { ProductEditor } from "@components/admin/product-editor";
import { ConfirmSubmit } from "@components/admin/confirm-submit";
import { deleteProduct } from "../actions";
import { cn } from "@lib/utills";

```



```

export const dynamic = "force-dynamic";

export default async function EditProductPage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const [product, categories, posts] = await Promise.all([
    getProductFullAdmin(id),
    listCategoriesAdmin(),
    listPostsForPicker(),
  ]);
  if (!product) notFound();

  return (
    <div className="flex flex-col gap-6">
      <Link
        href="/admin/products"
        className="text-muted-foreground hover:text-foreground inline-flex items-center gap-1.5 text-sm transition-colors"
      >
        <ArrowLeft className="size-4" />
        Products
      </Link>
      <div className="flex flex-wrap items-center gap-3">
        <h1 className="font-heading text-2xl font-bold tracking-tight">Edit {product.name}</h1>
        <span
          className={cn(
            "rounded-full px-2.5 py-0.5 text-xs font-medium ring-1 ring-inset",
            product.status === "published"
              ? "bg-emerald-500/12 text-emerald-500 ring-emerald-500/20"
              : "bg-muted text-muted-foreground ring-border",
          )}>
          {product.status === "published" ? "Published" : "Draft"}
        </span>
      </div>

      <ProductEditor product={product} categories={categories} posts={posts} />

      <div className="border-border max-w-3xl border-t pt-6">
        <form action={deleteProduct}>
          <input type="hidden" name="id" value={product.id} />
          <ConfirmSubmit
            message={`Delete "${product.name}"? This cannot be undone.`}
            className="border-destructive/30 text-destructive hover:bg-destructive/10 rounded-lg border px-4 py-2 text-sm font-medium transition-colors"
          >
            Delete product
          </ConfirmSubmit>
        </form>
      </div>
    </div>
  );
}

```

Step 8: Delete the old form and legacy block A

```
git rm src/components/admin/product-form.tsx
```

In `src/app/admin/(panel)/products/actions.ts`, **delete** `ProductFormState`, `parseForm`, `createProduct`, and `updateProduct` (everything from `export type ProductFormState` through the end of `updateProduct` — keep `toggleProductPublished`, which the list page still imports until Task 3), and remove the now-unused `redirect` import line (`import { redirect } from "next/navigation";`). Update the legacy comment block to read:

```
// -----
// Legacy --toggleProductPublished is removed in Task 3 with the list rewrite
// (it bypasses publish validation, which spec §3 forbids).
// -----
```

In `src/lib/admin-queries.ts`, **delete** the legacy `getProductAdmin` function (the three-line function plus its `// Legacy -- removed in Task 2... comment`). `ProductRow` stays imported (`AdminProductRow` extends it).

Step 9: Editor component test

Create `src/components/admin/product-editor.test.tsx`:

```
import { describe, it, expect, vi, beforeEach } from "vitest";
import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import { ProductEditor } from "@components/admin/product-editor";
import type { AdminProductFull } from "@lib/admin-queries";

const push = vi.fn();
const refresh = vi.fn();
vi.mock("next/navigation", () => ({
  useRouter: () => ({ push, refresh, back: vi.fn() }),
}));

const saveProductFull = vi.hoisted(() => vi.fn());
vi.mock("@/app/admin/(panel)/products/actions", () => ({
  saveProductFull,
  uploadProductImage: vi.fn(),
}));

const categories = [
  {
    id: "c1",
    name: "Productivity",
    slug: "productivity",
    sort_order: 10,
    created_at: "2026-06-01T00:00:00Z",
  },
];

function publishedProduct(): AdminProductFull {
  return {
    id: "p1",
    slug: "fluxboard",
    name: "FluxBoard",
    tagline: "Standups into shipped work.",
    summary: "A keyboard-first project board.",
    highlights: ["Fast", "Realtime", "Keyboard-first"],
    description: null,
    technical_details: null,
    cover_image: "https://cdn.example/cover.png",
    gallery: [],
    category_id: "c1",
    tech: [],
  };
}
```

```

    lifecycle: "live",
    year: null,
    live_url: null,
    repo_url: null,
    featured: false,
    sort_order: 10,
    status: "published",
    seo_description: null,
    og_image: null,
    published_at: "2026-07-01T00:00:00Z",
    created_at: "2026-06-01T00:00:00Z",
    updated_at: "2026-07-01T00:00:00Z",
    product_categories: { name: "Productivity" },
    product_features: [],
    product_blog_links: [],
  };
}

beforeEach(() => {
  saveProductFull.mockReset();
  push.mockReset();
});

describe("ProductEditor", () => {
  it("shows the four tier tabs", () => {
    render(<ProductEditor product={null} categories={categories} posts={[]} />);
    for (const label of ["Quick View", "Features", "Technical", "SEO & Links"]) {
      expect(screen.getByRole("tab", { name: new RegExp(label) })).toBeInTheDocument();
    }
  });

  it("auto-fills the slug from the name until the slug is edited", () => {
    render(<ProductEditor product={null} categories={categories} posts={[]} />);
    fireEvent.change(screen.getByLabelText(/Name/), { target: { value: "Flux Board 2" } });
    const slug = screen.getByLabelText(/Slug/) as HTMLInputElement;
    expect(slug.value).toBe("flux-board-2");
    fireEvent.change(slug, { target: { value: "custom" } });
    fireEvent.change(screen.getByLabelText(/Name/), { target: { value: "Renamed" } });
    expect((screen.getByLabelText(/Slug/) as HTMLInputElement).value).toBe("custom");
  });

  it("disables Publish and lists what's missing for an incomplete quick view", () => {
    render(<ProductEditor product={null} categories={categories} posts={[]} />);
    expect(screen.getByRole("button", { name: "Publish" })).toBeDisabled();
    expect(screen.getByText(/At least 3 highlights/)).toBeInTheDocument();
  });

  it("offers 'Unpublish & save as draft' when saving a published product fails validation", async () => {
    saveProductFull.mockResolvedValue({ ok: false, missing: ["Cover image"], error: null });
    render(<ProductEditor product={publishedProduct()} categories={categories} posts={[]} />);
    fireEvent.click(screen.getByRole("button", { name: "Save changes" }));
    await waitFor(() => {
      expect(
        screen.getByRole("button", { name: /Unpublish & save as draft instead/ }),
      ).toBeInTheDocument();
    });
  });
});

```

```
});
});
```

Step 10: Run the new tests

Run: `npx vitest run src/components/admin/product-editor.test.tsx` → PASS (4 tests).

Step 11: Green bar

Run: `npm run typecheck && npm run lint && npm test && npm run build` → all green.

Step 12: Commit

```
git add package.json package-lock.json src/components/admin/form-styles.ts src/
components/admin/sortable-list.tsx src/components/admin/image-upload.tsx src/
components/admin/related-posts-picker.tsx src/components/admin/product-editor.tsx src
/components/admin/product-editor.test.tsx "src/app/admin/(panel)/products/new/page.
tsx" "src/app/admin/(panel)/products/[id]/page.tsx" "src/app/admin/(panel)/products/
actions.ts" src/lib/admin-queries.ts
git commit -m "feat: tiered product editor --quick view/features/technical/seo tabs,
publish validation with unpublish escape, uploads"
```

7 Task 3: List view + categories manager + preview route

Files

- Create: `src/components/admin/products-table.tsx`
- Modify: `src/app/admin/(panel)/products/page.tsx` (whole-file replace)
- Modify: `src/app/admin/(panel)/products/actions.ts` (remove `toggleProductPublished`)
- Create: `src/app/admin/(panel)/products/categories/page.tsx`, `src/components/admin/categories-`
- Create: `src/app/admin/(panel)/products/[id]/preview/page.tsx`, `src/components/admin/preview-q`

Interfaces

- Consumes: `listProductsAdmin`, `listCategoriesAdmin`, `countProductsByCategory`, `getProductFullAdmin` (Task 1); `reorderProducts`, `duplicateProduct`, `deleteProduct`, `category actions` (Task 1); `SortableList/SortableItem`, `form-styles` (Task 2); **unchanged public exports** `toProductDetail` (`src/lib/queries.ts`), `ProductDetailView`, `ProductQuickView` (`ProductQuickView` needs `product/open/onClose` — the small client wrapper supplies them).
- Produces: `<ProductsTable products categories />`; `<CategoriesManager categories={CategoryWithCount []} />`; `<PreviewQuickView product={Product} />`; **routes** `/admin/products/categories` and `/admin/products/[id]/preview (+ ?tier=quick)`.

Step 1: Create `src/components/admin/products-table.tsx`

```
"use client";

import { useMemo, useState, useTransition } from "react";
import Image from "next/image";
import Link from "next/link";
import { useRouter } from "next/navigation";
import { Copy, Pencil, Star } from "lucide-react";
```

```

import type { ProductCategoryRow } from "@lib/supabase/types";
import type { AdminProductRow } from "@lib/admin-queries";
import {
  deleteProduct,
  duplicateProduct,
  reorderProducts,
} from "@app/admin/(panel)/products/actions";
import { ConfirmSubmit } from "@components/admin/confirm-submit";
import { SortableList, SortableItem } from "@components/admin/sortable-list";
import { fieldClass } from "@components/admin/form-styles";
import { cn } from "@lib/utils";

const REORDER_DISABLED_HINT = "Clear the filters to reorder --positions are global.";

function ProductRowItem({
  product: p,
  dragDisabled,
  onDuplicate,
  busy,
}): {
  product: AdminProductRow;
  dragDisabled: boolean;
  onDuplicate: (id: string) => void;
  busy: boolean;
}) {
  return (
    <SortableItem
      id={p.id}
      disabled={dragDisabled}
      handleTitle={dragDisabled ? REORDER_DISABLED_HINT : "Drag to reorder"}
      className="border-border bg-card items-center rounded-xl border p-3"
    >
      <div className="flex flex-wrap items-center gap-3">
        <div className="bg-muted relative aspect-[16/9] w-20 shrink-0 overflow-hidden rounded-md">
          {p.cover_image ? (
            <Image src={p.cover_image} alt="" fill sizes="5rem" className="object-cover" />
          ) : null}
        </div>
        <div className="min-w-0 flex-1">
          <div className="flex items-center gap-1.5">
            {p.featured ? (
              <Star className="size-3.5 shrink-0 fill-amber-400 text-amber-400" aria-label="Featured" />
            ) : null}
            <span className="text-foreground truncate text-sm font-medium">{p.name}</span>
          </div>
          <div className="text-muted-foreground truncate font-mono text-xs">{p.slug}</div>
        </div>
        <span className="text-muted-foreground hidden w-28 truncate text-xs sm:inline">
          {p.product_categories?.name ?? "--"}
        </span>
        <span className="text-muted-foreground w-12 text-xs capitalize">{p.lifecycle}</span>
      </div>
      <span
        className={cn(
          "rounded-full px-2.5 py-0.5 text-xs font-medium ring-1 ring-inset",
          p.status === "published"
        )}
      >

```

```

        ? "bg-emerald-500/12 text-emerald-500 ring-emerald-500/20"
        : "bg-muted text-muted-foreground ring-border",
    ))
  >
    {p.status === "published" ? "Published" : "Draft"}
  </span>
  <div className="flex items-center gap-2">
    <Link
      href={` /admin/products/${p.id}`}
      className="border-border text-muted-foreground hover:text-foreground inline-
        flex items-center gap-1 rounded-md border px-2.5 py-1 text-xs transition-
        colors"
    >
      <Pencil className="size-3.5" />
      Edit
    </Link>
    <button
      type="button"
      disabled={busy}
      onClick={() => onDuplicate(p.id)}
      className="border-border text-muted-foreground hover:text-foreground inline-
        flex items-center gap-1 rounded-md border px-2.5 py-1 text-xs transition-
        colors disabled:opacity-50"
    >
      <Copy className="size-3.5" />
      Duplicate
    </button>
    <form action={deleteProduct}>
      <input type="hidden" name="id" value={p.id} />
      <ConfirmSubmit
        message={`Delete "${p.name}"? This cannot be undone.`}
        className="border-destructive/30 text-destructive hover:bg-destructive/10
          rounded-md border px-2.5 py-1 text-xs transition-colors"
      >
        Delete
      </ConfirmSubmit>
    </form>
  </div>
</div>
</SortableItem>
);
}

/**
 * Admin list (spec §3): featured group first (mirrors the public order --a drag
 * can never cross the featured boundary, because publicly it couldn't change
 * anything), status/category filters, drag reorder persisting sort_order,
 * disabled while any filter is active.
 */
export function ProductsTable({
  products,
  categories,
}): {
  products: AdminProductRow[];
  categories: ProductCategoryRow[];
} {
  const router = useRouter();
  const [statusFilter, setStatusFilter] = useState("all");
  const [categoryFilter, setCategoryFilter] = useState("all");
  const [error, setError] = useState<string | null>(null);

```

```

const [pending, startTransition] = useTransition();

// Optimistic row order; server order (featured DESC, sort_order ASC) seeds it.
const [order, setOrder] = useState(() => products.map((p) => p.id));
// Re-sync when the server sends a different row set (duplicate/delete/refresh).
// Setting state during render is React's sanctioned derived-state pattern.
const serverIds = products.map((p) => p.id).join("|");
const [lastServerIds, setLastServerIds] = useState(serverIds);
if (serverIds !== lastServerIds) {
  setLastServerIds(serverIds);
  setOrder(products.map((p) => p.id));
}

const byId = useMemo(() => new Map(products.map((p) => [p.id, p])), [products]);
const ordered = order
  .map((id) => byId.get(id))
  .filter((p): p is AdminProductRow => Boolean(p));

const filtersActive = statusFilter !== "all" || categoryFilter !== "all";
const visible = ordered.filter(
  (p) =>
    (statusFilter === "all" || p.status === statusFilter) &&
    (categoryFilter === "all" || p.category_id === categoryFilter),
);

const featuredRows = visible.filter((p) => p.featured);
const normalRows = visible.filter((p) => !p.featured);
const dragDisabled = filtersActive || pending;

function persist(nextGroupIds: string[], group: "featured" | "normal") {
  // Only reachable with filters off, so featured + normal cover every row.
  const featIds = group === "featured" ? nextGroupIds : featuredRows.map((p) => p.id);
  const normIds = group === "normal" ? nextGroupIds : normalRows.map((p) => p.id);
  const next = [...featIds, ...normIds];
  setOrder(next);
  setError(null);
  startTransition(async () => {
    const result = await reorderProducts(next);
    if (!result.ok) setError(result.error);
    router.refresh();
  });
}

function duplicate(id: string) {
  setError(null);
  startTransition(async () => {
    const result = await duplicateProduct(id);
    if (!result.ok) setError(result.error);
    router.refresh();
  });
}

return (
  <div className="flex flex-col gap-4">
    {error ? (
      <p className="border-destructive/30 bg-destructive/10 text-destructive rounded-lg border px-3.5 py-2.5 text-sm">
        {error}
      </p>
    ) : null}
  </div>
);

```

```

<div className="flex flex-wrap items-center gap-3">
  <select
    value={statusFilter}
    onChange={(e) => setStatusFilter(e.target.value)}
    aria-label="Filter by status"
    className={cn(fieldClass, "w-auto")}
  >
    <option value="all">All statuses</option>
    <option value="draft">Draft</option>
    <option value="published">Published</option>
  </select>
  <select
    value={categoryFilter}
    onChange={(e) => setCategoryFilter(e.target.value)}
    aria-label="Filter by category"
    className={cn(fieldClass, "w-auto")}
  >
    <option value="all">All categories</option>
    {categories.map((c) => (
      <option key={c.id} value={c.id}>
        {c.name}
      </option>
    ))}
  </select>
  {filtersActive ? (
    <p className="text-muted-foreground text-xs" title={REORDER_DISABLED_HINT}>
      Reordering is off while filters are active --positions are global.
    </p>
  ) : null}
</div>

{visible.length === 0 ? (
  <p className="border-border bg-card text-muted-foreground rounded-2xl border p-8 text-sm">
    No products match these filters.
  </p>
) : (
  <div className="flex flex-col gap-4">
    {featuredRows.length > 0 ? (
      <div className="flex flex-col gap-2">
        <p className="text-muted-foreground text-xs font-medium tracking-wide uppercase">
          Featured --always first on the public grid
        </p>
        <SortableList
          ids={featuredRows.map((p) => p.id)}
          disabled={dragDisabled}
          onReorder={(ids) => persist(ids, "featured")}
        >
          <div className="flex flex-col gap-2">
            {featuredRows.map((p) => (
              <ProductRowItem
                key={p.id}
                product={p}
                dragDisabled={dragDisabled}
                onDuplicate={duplicate}
                busy={pending}
              />
            ))}
          </div>
        </div>
      </div>
    ) : (
      <p>No featured products</p>
    )}
  </div>
) : null}

```



```

        </div>
      </SortableList>
    </div>
  ) : null}

  <div className="flex flex-col gap-2">
    {featuredRows.length > 0 ? (
      <p className="text-muted-foreground text-xs font-medium tracking-wide uppercase">
        Everything else
      </p>
    ) : null}
    <SortableList
      ids={normalRows.map((p) => p.id)}
      disabled={dragDisabled}
      onReorder={(ids) => persist(ids, "normal")}
    >
      <div className="flex flex-col gap-2">
        {normalRows.map((p) => (
          <ProductRowItem
            key={p.id}
            product={p}
            dragDisabled={dragDisabled}
            onDuplicate={duplicate}
            busy={pending}
          />
        ))}
      </div>
    </SortableList>
  </div>
</div>
  )}
</div>
);
}

```

Step 2: Replace `src/app/admin/(panel)/products/page.tsx`

```

import Link from "next/link";
import { Plus, Tags } from "lucide-react";
import { listProductsAdmin, listCategoriesAdmin } from "@lib/admin-queries";
import { ProductsTable } from "@components/admin/products-table";
import { buttonVariants } from "@components/ui/button";
import { cn } from "@lib/utills";

export const dynamic = "force-dynamic";

export default async function AdminProductsPage() {
  const [products, categories] = await Promise.all([listProductsAdmin(), listCategoriesAdmin()]);

  return (
    <div className="flex flex-col gap-6">
      <div className="flex flex-wrap items-center justify-between gap-4">
        <div>
          <h1 className="font-heading text-2xl font-bold tracking-tight">Products</h1>
          <p className="text-muted-foreground text-sm">
            {products.length} in the catalog --drag rows to set the public order
          </p>
        </div>
      </div>
    </div>
  );
}

```

```

    </p>
  </div>
  <div className="flex items-center gap-2">
    <Link
      href="/admin/products/categories"
      className={cn(buttonVariants({ variant: "outline" }), "gap-1.5")}
    >
      <Tags className="size-4" />
      Categories
    </Link>
    <Link href="/admin/products/new" className={cn(buttonVariants(), "gap-1.5")}>
      <Plus className="size-4" />
      New product
    </Link>
  </div>
</div>

{products.length === 0 ? (
  <p className="border-border bg-card text-muted-foreground rounded-2xl border p-8 text-sm">
    No products yet. Create your first one.
  </p>
) : (
  <ProductsTable products={products} categories={categories} />
)}
</div>
);
}

```

Step 3: Remove the last legacy action

In `src/app/admin/(panel)/products/actions.ts`, delete the legacy comment block and the whole `toggleProductPublished` function (publishing now only flows through `saveProductFull`'s validated intents). Nothing imports it anymore — `npm run typecheck` confirms.

Step 4: Create `src/components/admin/categories-manager.tsx`

```

"use client";

import { useState, useTransition } from "react";
import { useRouter } from "next/navigation";
import { Check, Plus, Trash2 } from "lucide-react";
import type { ProductCategoryRow } from "@/lib/supabase/types";
import {
  createCategory,
  deleteCategory,
  renameCategory,
  reorderCategories,
  type CategoryActionResult,
} from "@/app/admin/(panel)/products/categories/actions";
import { SortableList, SortableItem } from "@/components/admin/sortable-list";
import { fieldClass } from "@/components/admin/form-styles";
import { buttonVariants } from "@/components/ui/button";
import { cn } from "@/lib/utils";

export type CategoryWithCount = ProductCategoryRow & { productCount: number };

export function CategoriesManager({ categories }: { categories: CategoryWithCount[] }) {

```

```

const router = useRouter();
const [error, setError] = useState<string | null>(null);
const [newName, setNewName] = useState("");
const [pending, startTransition] = useTransition();

// Optimistic order + rename drafts, re-synced when server data changes
// (same derived-state pattern as ProductsTable).
const serverKey = categories.map((c) => `${c.id}:${c.name}`).join("|");
const [lastServerKey, setLastServerKey] = useState(serverKey);
const [order, setOrder] = useState(() => categories.map((c) => c.id));
const [names, setNames] = useState<Record<string, string>>(() =>
  Object.fromEntries(categories.map((c) => [c.id, c.name])),
);
if (serverKey !== lastServerKey) {
  setLastServerKey(serverKey);
  setOrder(categories.map((c) => c.id));
  setNames(Object.fromEntries(categories.map((c) => [c.id, c.name])));
}

const byId = new Map(categories.map((c) => [c.id, c]));
const ordered = order
  .map((id) => byId.get(id))
  .filter((c): c is CategoryWithCount => Boolean(c));

function run(action: () => Promise<CategoryActionResult>) {
  setError(null);
  startTransition(async () => {
    const result = await action();
    if (!result.ok) setError(result.error);
    router.refresh();
  });
}

return (
  <div className="flex max-w-2xl flex-col gap-5">
    {error ? (
      <p className="border-destructive/30 bg-destructive/10 text-destructive rounded-lg border px-3.5 py-2.5 text-sm">
        {error}
      </p>
    ) : null}

    <form
      onSubmit={(e) => {
        e.preventDefault();
        const name = newName.trim();
        if (!name) return;
        setNewName("");
        run(() => createCategory(name));
      }}
      className="flex items-center gap-2"
    >
      <input
        value={newName}
        onChange={(e) => setNewName(e.target.value)}
        placeholder="New category name"
        aria-label="New category name"
        className={fieldClass}
      />

```

```

<button type="submit" disabled={pending} className={cn(buttonVariants(), "shrink
  -0 gap-1.5")}>
  <Plus className="size-4" />
  Add
</button>
</form>

{ordered.length === 0 ? (
  <p className="text-muted-foreground text-sm">No categories yet.</p>
) : (
  <SortableList
    ids={ordered.map((c) => c.id)}
    disabled={pending}
    onReorder={(ids) => {
      setOrder(ids);
      run(() => reorderCategories(ids));
    }}
  >
    <div className="flex flex-col gap-2">
      {ordered.map((c) => {
        const draft = names[c.id] ?? c.name;
        const dirty = draft.trim() !== c.name && draft.trim() !== "";
        return (
          <SortableItem
            key={c.id}
            id={c.id}
            disabled={pending}
            className="border-border bg-card items-center rounded-xl border p-3"
          >
            <div className="flex items-center gap-3">
              <input
                value={draft}
                onChange={(e) => setNames({ ...names, [c.id]: e.target.value })}
                aria-label={`Rename ${c.name}`}
                className={fieldClass}
              />
              {dirty ? (
                <button
                  type="button"
                  disabled={pending}
                  onClick={() => run(() => renameCategory(c.id, draft))}
                  className="border-border text-muted-foreground hover:text-
                    foreground inline-flex shrink-0 items-center gap-1 rounded-md
                    border px-2.5 py-1.5 text-xs transition-colors"
                >
                  <Check className="size-3.5" />
                  Save
                </button>
              ) : null}
            <span className="text-muted-foreground w-20 shrink-0 text-right text-
              xs">
              {c.productCount} product{c.productCount === 1 ? "" : "s"}
            </span>
            <button
              type="button"
              disabled={pending || c.productCount > 0}
              title={
                c.productCount > 0
                  ? "In use --reassign its products in the editor first."
                  : "Delete category"
              }
            >

```

```

        }
        onClick={() => {
          if (window.confirm(`Delete "${c.name}"?`)) run(() => deleteCategory(c.id));
        }}
        className="text-muted-foreground hover:text-destructive shrink-0 p-1.5 transition-colors disabled:opacity-40"
      >
        <Trash2 className="size-4" />
      </button>
    </div>
  </SortableItem>
);
  })}
</div>
</SortableList>
)}
</div>
);
}

```

Step 5: Create `src/app/admin/(panel)/products/categories/page.tsx`

```

import Link from "next/link";
import { ArrowLeft } from "lucide-react";
import { listCategoriesAdmin, countProductsByCategory } from "@lib/admin-queries";
import { CategoriesManager } from "@components/admin/categories-manager";

export const dynamic = "force-dynamic";

export default async function CategoriesPage() {
  const [categories, counts] = await Promise.all([
    listCategoriesAdmin(),
    countProductsByCategory(),
  ]);
  const rows = categories.map((c) => ({ ...c, productCount: counts[c.id] ?? 0 }));

  return (
    <div className="flex flex-col gap-6">
      <Link
        href="/admin/products"
        className="text-muted-foreground hover:text-foreground inline-flex items-center gap-1.5 text-sm transition-colors"
      >
        <ArrowLeft className="size-4" />
        Products
      </Link>
      <div>
        <h1 className="font-heading text-2xl font-bold tracking-tight">Product categories
        </h1>
        <p className="text-muted-foreground text-sm">
          This order is the filter-pill order on /products. Categories in use can't be
          deleted --
          reassign their products first.
        </p>
      </div>
      <CategoriesManager categories={rows} />
    </div>
  );
}

```

```
);
}
```

Step 6: Create `src/components/admin/preview-quick-view.tsx`

```
"use client";

import { useRouter } from "next/navigation";
import type { Product } from "@lib/content";
import { ProductQuickView } from "@components/product-quick-view";

/**
 * Mounts the REAL tier-1 dialog open over the preview route. ProductQuickView
 * needs open/onClose (its parent normally owns URL sync) --here closing it
 * (Esc / backdrop / close event) steps back in history, i.e. to the tier-2
 * preview or the editor, wherever the admin came from.
 */
export function PreviewQuickView({ product }: { product: Product }) {
  const router = useRouter();
  return <ProductQuickView product={product} open onClose={() => router.back()} />;
}
```

Step 7: Create `src/app/admin/(panel)/products/[id]/preview/page.tsx`

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { ArrowLeft } from "lucide-react";
import { getProductFullAdmin } from "@lib/admin-queries";
import { toProductDetail } from "@lib/queries";
import { ProductDetailView } from "@components/product-detail-view";
import { PreviewQuickView } from "@components/admin/preview-quick-view";
import { cn } from "@lib/utils";

export const dynamic = "force-dynamic";

/**
 * Admin-only preview (spec §3): the ACTUAL public components rendered from the
 * last-saved draft --admin client fetch, then the same toProductDetail mapping
 * the public routes use. Inside the guarded (panel) layout. Draft related
 * posts are filtered out exactly like anon RLS does publicly, so the preview
 * never shows a chip the public page would hide.
 */
export default async function ProductPreviewPage({
  params,
  searchParams,
}: {
  params: Promise<{ id: string }>;
  searchParams: Promise<{ tier?: string }>;
}) {
  const { id } = await params;
  const { tier } = await searchParams;
  const row = await getProductFullAdmin(id);
  if (!row) notFound();

  const detail = toProductDetail({
    ...row,
    product_blog_links: row.product_blog_links.filter(
```

```

        (1) => 1.blog_posts?.status === "published",
    ),
    });

return (
    <div className="flex flex-col gap-4">
        <div className="border-border bg-card/60 flex flex-wrap items-center justify-
            between gap-3 rounded-xl border px-4 py-3 text-sm">
            <div className="flex flex-wrap items-center gap-3">
                <Link
                    href={`'/admin/products/${id}'`}
                    className="text-muted-foreground hover:text-foreground inline-flex items-
                        center gap-1.5 transition-colors"
                >
                    <ArrowLeft className="size-4" />
                    Back to editor
                </Link>
                <span className="text-muted-foreground">
                    Previewing the last saved {row.status === "published" ? "version" : "draft"}.
                    Links
                    inside the preview go to the public site and only work once published.
                </span>
            </div>
            <div className="flex items-center gap-1">
                <Link
                    href={`'/admin/products/${id}/preview'`}
                    className={cn(
                        "rounded-md px-3 py-1.5 text-xs font-medium transition-colors",
                        tier !== "quick"
                            ? "bg-muted text-foreground"
                            : "text-muted-foreground hover:text-foreground",
                    )}
                >
                    Technical page
                </Link>
                <Link
                    href={`'/admin/products/${id}/preview?tier=quick'`}
                    className={cn(
                        "rounded-md px-3 py-1.5 text-xs font-medium transition-colors",
                        tier === "quick"
                            ? "bg-muted text-foreground"
                            : "text-muted-foreground hover:text-foreground",
                    )}
                >
                    Quick View
                </Link>
            </div>
        </div>

        {tier === "quick" ? (
            <PreviewQuickView product={detail} />
        ) : (
            <div className="border-border overflow-hidden rounded-2xl border">
                <ProductDetailView detail={detail} />
            </div>
        )}
    </div>
);
}

```

Note: `ProductDetail` extends `Product`, so detail feeds both components; `toProduct` (inside `toProductDetail`) builds the quick view's related chips from the same filtered links. `AdminProductFull` is structurally a `ProductRowJoined` (its `AdminLinkJoin.blog_posts` carries an extra status field, which the mapper ignores).

Step 8: Green bar

Run: `npm run typecheck && npm run lint && npm test && npm run build` → all green. Build output lists the two new routes: `/admin/products/categories` and `/admin/products/[id]/preview`.

Step 9: Commit

```
git add src/components/admin/products-table.tsx src/components/admin/categories-manager.tsx src/components/admin/preview-quick-view.tsx "src/app/admin/(panel)/products/page.tsx" "src/app/admin/(panel)/products/actions.ts" "src/app/admin/(panel)/products/categories/page.tsx" "src/app/admin/(panel)/products/[id]/preview/page.tsx"
git commit -m "feat: admin products list with grouped drag reorder, categories manager, both-tier draft preview"
```

8 Task 4: Visual QA, plan PDF twin, PR

(Controller-level task — needs the local server, admin credentials, and human-visible artifacts.)

Step 1: Boot

`npm run build green` → `npm run dev` → sign in at `/admin/login`.

Step 2: Validation walkthrough (the spec §3 contract, end to end)

1. **Create draft:** New product → type a name, watch the slug auto-fill; edit the slug once and confirm it stops following the name. Add only **2** highlights → **Publish is disabled** and the footer lists the missing fields (must include “At least 3 highlights”, “Cover image”, “Category”).
2. **Fill:** third highlight, tagline, summary, upload a cover (< 5 MB), pick a category → Publish enables. Click **Save draft** (lands on the product's editor).
3. **Preview both tiers:** Preview → technical page renders with feature blocks in editor order; switch to **Quick View** → the real dialog opens; **Esc returns to the tier-2 preview** (history.back).
4. **Publish:** Publish → `/products` shows the card immediately (revalidatePath, no redeploy) and `/products/[slug]` renders.
5. **Guarded edit:** reopen the published product, **remove the cover image** → Save changes → blocked banner names “Cover image” and offers **Unpublish & save as draft instead** → click it → product is Draft, the public detail page 404s and the card is gone.
6. **Duplicate:** Duplicate a product → new Draft row “... (copy)” with slug `...-copy` at the **end** of the list, not featured, features + related links present in its editor; duplicate the same product again → `...-copy-2`.
7. **Reorder:** drag a row → `/products` order follows (featured block stays first). Set any filter → drag handles disable and the “positions are global” hint appears.
8. **Categories:** add one → it appears in the editor select + as a pill on `/products` once a published product uses it; rename it; try deleting one **in use** → guarded error tells you to reassign; reassign the product, delete succeeds; drag-reorder categories → pill order on `/products` follows.

9. **Upload guards:** try a > 5 MB image and a .txt file → inline errors, nothing uploaded.

Step 3: Screenshots

(dark + light): admin list (featured group + filters), each editor tab, the blocked-save banner with the unpublish escape, preview both tiers, categories page. Attach to the PR.

Step 4: Plan PDF twin

Generate docs/superpowers/plans/2026-07-05-products-admin.tex + .pdf (brand LaTeX template, lstlistings for code) and commit (docs: prefix).

Step 5: PR

Push feat/products-admin; open a PR titled “feat: products admin — tiered editor, drag reorder, categories, preview” summarizing spec §3 coverage, the QA checklist results, and screenshots. main is PR-protected — 1 review required.

Step 6: After merge + deploy

Repeat steps 2.4 and 2.9 against production (Vercel + Supabase Cloud); confirm uploads land in the product-images bucket.

9 Self-Review (done)

- **Spec §3 coverage:** list view — cover thumb / status badge / lifecycle / featured star (done), status + category filters (done), drag-to-reorder persisting `sort_order` (done), disabled while filtered with the “positions are global” explanation (done), featured grouped first with drags never crossing the boundary (done), Duplicate (done). Editor — four tabs mirroring the public tiers (done); every field labeled with its public destination (the research pattern) (done); auto-slug with manual override (done); highlights add/remove/drag with the “plain language, no jargon” helper (done); category select + Manage categories link (done); cover upload with preview (done); features repeating rows (image/title/explanation, add/remove/ drag, optional-at-publish) (done); technical tab (description, technical_details, tech list, gallery multi-upload with remove, repo URL, year) (done); SEO & Links (seo_description “(search results)”, og_image, related-posts picker with search/multi-select/drag-order/draft badge) (done). Publish validation on publish **and** on saves of published rows, with the Unpublish & save as draft escape, errors naming the missing quick-view fields (done) (helper unit-tested; enforced server-side; mirrored client-side for the disabled Publish button). Preview renders the actual ProductQuickView + ProductDetailView from the last-saved draft via admin fetch + toProductDetail (done) (save-first noted in the editor flow; quick-view wrapper closes via `history.back()`). Duplicate semantics per spec (draft, -copy retry via unique-violation code 23505, featured off, end of list, features + links copied, image URLs by reference) (done). Categories add/rename/delete-guarded/reorder (done) (guard is app-level — the FK is ON DELETE SET NULL, so the DB alone wouldn’t block). Uploads to product-images via the server client, image/*, ≤ 5 MB (done).
- **Global constraints honored:** only @dnd-kit added; public components/queries untouched (preview imports them); every mutation revalidates /, /products, detail path(s), /admin/products; no service_role; Conventional Commits; green bar closes every task.
- **Task-boundary compile check:** Task 1 keeps getProductAdmin + the legacy form/toggle actions so the untouched pages still build; Task 2 removes the form-era exports with the

pages that used them; Task 3 removes `toggleProductPublished` with the list rewrite (it bypassed publish validation — spec §3 forbids that path anyway). Each task ends green in isolation.

- **Placeholder scan:** every code step is complete file content or an exact named deletion — no “similar to”, no “add validation here”.
- **Type consistency:** `AdminProductFull` is structurally accepted by `toProduct/toProductDetail` (extra `blog_post_id/status` join fields are ignored); `ProductDetail` extends `Product` feeds both preview tiers from one mapping call; `saveProductFull`’s discriminated `SaveProductResult` drives the editor’s banner + escape hatch; keyed client rows (`HighlightItem/FeatureItem`) exist only in the editor — payload strips keys.
- **Known judgment calls (flagged for review):** duplicate also suffixes the *name* with “ (copy)” so the two rows are distinguishable in the list (spec only mandates the slug suffix); after saving a **new** product the editor navigates to `/admin/products/[id]` (so Preview/Publish are one click away) while existing products return to the list; `next.config.ts` uses `experimental.serverActions.b` with an explicit fallback note if Next 16 has promoted the key; gallery images upload/remove without drag-ordering (spec lists drag only for highlights, features, related posts, list rows, and categories).